A COST-EFFECTIVE EDGE COMPUTING FRAMEWORK FOR VIDEO PROCESSING

Project thesis submitted to
Visvesvaraya National Institute of Technology, Nagpur
in partial fulfillment of the requirements for the award of
the degree

Bachelor of Technology
in
Electronics and Communication Engineering

by

Sidharth Dinesh (BT18ECE066)
Apurva Umredkar (BT18ECE086)

under the guidance of

Dr. Ashwin G. Kothari



Department of Electronics and Communication Engineering
Visvesvaraya National Institute of Technology
Nagpur 440 010

2021 - 2022

A COST-EFFECTIVE EDGE COMPUTING FRAMEWORK FOR VIDEO PROCESSING

Project thesis submitted to
Visvesvaraya National Institute of Technology, Nagpur
in partial fulfillment of the requirements for the award of
the degree

Bachelor of Technology
in
Electronics and Communication Engineering

by

Sidharth Dinesh (BT18ECE066)
Apurva Umredkar (BT18ECE086)

under the guidance of

Dr. Ashwin G. Kothari



Department of Electronics and Communication Engineering
Visvesvaraya National Institute of Technology
Nagpur 440 010

2021 - 2022

© Visvesvaraya National Institute of Technology (VNIT) 2022

Department of Electronics and Communication Engineering
Visvesvaraya National Institute of Technology, Nagpur

Declaration

We, "Sidharth Dinesh" and "Apurva Umredkar" hereby declare that the project work titled **"Dynamic Decision-Making Engine For Cost-Effective Edge Computing Framework For Video Processing"** is carried out by us in the Department of Electronics and Communication Engineering of VNIT, Nagpur, India. The work is original and has not been submitted earlier as a whole or in part for the award of any degree/diploma at this or any other Institution/University.

Sidharth Dinesh (BT18ECE066)

Apurva Umredkar (BT18ECE086)

Date: 22-04-2022

Certificate

This is to certify that the thesis titled **"Dynamic Decision-Making Engine For Cost-Effective Edge Computing Framework For Video Processing"** submitted by **Sidharth Dinesh** and **Apurva Umredkar** in partial fulfillment of requirements for the award of **Bachelor of Technology in the Department of Electronics and Communication Engineering** of VNIT, Nagpur. The work is comprehensive, complete, and fit for evaluation.

Dr. Ashwin G. Kothari
Associate Professor,
Department of Electronics and
Communication Engineering,
VNIT, Nagpur

Dr. A.G. Keskar
Professor and Head,
Department of Electronics and
Communication Engineering
VNIT, Nagpur
Date: 22-04-2022

ACKNOWLEDGEMENTS

Our sincere gratitude to our guide, Dr. Ashwin G. Kothari, for his constant support and encouragement, without which the project would not have been possible. We would like to thank Mr. Prasanna Deshpande, Consultant at TCS Nagpur and PhD Research Scholar at IIIT Nagpur, for offering us this industry-collaborated project. Our heartfelt gratitude to them for offering all of the facilities and assisting us in overcoming obstacles.

We would also like to acknowledge all the faculties of the ECE department for lending a hand at all times. We sincerely thank our project evaluators Dr. Abhay S. Gandhi and Dr. Kishor D. Kulat for their valuable comments and suggestions. We thank Dr. A.G. Keskar, Head of the ECE department for giving us an opportunity for the project.

Date: 22-04-2022

Sidharth Dinesh (BT18ECE066)

Apurva Umredkar (BT18ECE086)

ABSTRACT

In this thesis, the existing edge computing algorithms are introduced and a cost-effective algorithm for reducing the computational overhead among various edge servers in an existing network infrastructure for an optimum state of operation has been proposed, which will prove beneficial for remote areas of third world countries where limited resources and the minimization of the cost of operation are essential factors. The time required to process different video streams or frames while running the proposed algorithm, named Dynamic Decision Making Engine or DDME, remains the same or improved compared to other algorithms while keeping the accuracy of the video analytics application more or less the same.

The working principle of the DDME is reallocating the incoming video packets from one server to another based on their utilization percentage. The amount of utilization is determined by the size of the video buffer received at the server, which is majorly affected by the network conditions available in the source environment. On determining the utilization percentage, DDME then divides the servers into two categories. The servers with more than 80% utilization are termed as over-utilized while the ones below 40% are termed under-utilized. The servers are ordered and paired on the basis of these percentages, and the video packets are transferred from the over-utilized to under-utilized servers, bringing the entire network to a balanced processing capacity.

Such load-balancing of an edge computing infrastructure speeds up video analytics applications. For this study, the YOLO CNN algorithm, commonly deployed in object detection tasks, has been used for benchmarking algorithms against existing works in terms of speed and accuracy.

LIST OF FIGURES

1.1: Typical cloud computing environment.
1.2: Setup of an edge computing environment.
3.1: Working principle of the DeepDecision algorithm.
3.2: Mathematical description of the DeepDecision algorithm.
3.3: NGROK window post successful connection between client and server.
3.4: Sample output of object detection using YOLO CNN.
3.5: Processing time required for various video frame resolutions.
3.6: Energy consumed per frame by the device for various video frame resolutions.
3.7: Variation of accuracy of YOLO CNNs with input resolution and bitrates.
3.8: Trends in total system latency vs network bandwidth.
4.1 Various applications of video analytics.
4.2: Pipeline of a video analytics application
4.3 Edge-cloud interworking mechanism.
4.4 The processing flow for the task offloading-job balancing algorithm.
4.5: Greedy Task Offload Scheduling Algorithm.
4.6: Greedy Job Load Balancing algorithm.
4.7: Average service latency and average total monetary cost w.r.t. deadline.
4.8: Average service latency and average total monetary cost w.r.t. the number of concurrent jobs.
4.9: Average service latency and average total monetary cost w.r.t. average RTT (round trip time).
5.1 System architecture with DDME server handling a variable number of edge agents and video sources.
5.2: Data Reallocation Algorithm.
5.3: Dynamic Decision Making Engine Algorithm.
5.4: Example of pairing edge nodes based on resource utilization.
5.5: Optimization of edge nodes with DDME.
5.6: Performance evaluation for average service latency w.r.t. deadline.
5.7: Comparison of DDME with the framework proposed by Ahn et al.

LIST OF TABLES

3.1: Table of Notations.

3.2: Performance comparison of TinyYOLOv3 vs BigYOLOv3.

5.1: Video frame resolution vs required streaming bandwidth & data buffer size per frame.

5.2: Table of Notations.

LIST OF ABBREVIATIONS

COCO	Common Objects in Context
CNN	Convolutional Neural Network
DDME	Dynamic Decision Making Engine
EI	Edge Intelligence
FLOPS	Floating Point Operations
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	Mean Average Precision
QoS	Quality of Service
YOLO	You Only Look Once
IDE	Integrated Development Environment

LIST OF PUBLICATIONS

1. Prasanna Deshpande, Sidharth Dinesh, Apurva Umredkar and Ashwin G Kothari. “Dynamic Decision Making Engine for Cost-effective Edge Computing Framework for Video Processing”, Journal of Network and Systems Management (submitted and under review)

TABLE OF CONTENTS

ABSTRACT i

LIST OF FIGURES ii

LIST OF TABLES iii

LIST OF ABBREVIATIONS iv

LIST OF PUBLICATIONS v

CHAPTER 1: INTRODUCTION 1

1.1 Cloud Computing 1

1.1.1 Challenges in Cloud Computing 2

1.2 Edge Computing 2

1.2.1 Advantages of Edge Computing 3

1.3 The Need For Load Balancing 3

CHAPTER 2: LITERATURE SURVEY 6

2.1 Edge Video Analytics for Public Safety: A Review. 6

2.2 Video Caching, Analytics, and Delivery at the Wireless Edge: A Survey and Future Directions. 6

2.3 Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. 6

2.4 Edge Computing Framework for Cooperative Video Processing in Multimedia IoT Systems 7

2.5 Computation Offloading Toward Edge Computing 7

2.6 DeepDecision: A Mobile Deep Learning Framework for Edge Video Analytics. 7

2.7 A Cost-Effective Deadline-Constrained Dynamic Scheduling Algorithm for Scientific Workflows in a Cloud Environment. 8

2.8 A Novel Edge-Cloud Interworking Framework in the Video Analytics of the Internet of Things. 8

2.9 Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. 8

CHAPTER 3: PRELIMINARY WORK 9

3.1 The DeepDecision Algorithm 10

3.2 Test Setup Used for Replicating DeepDecision 13

3.2.1 Hardware Setup 13

3.2.2 Connection between the client and server 14

3.2.3 Deep Learning Models 14

3.3 Simulation Results 16

3.4 Summary 18

CHAPTER 4: THE TASK OFFLOADING-JOB BALANCING ALGORITHM 20

4.1 Video analytics & its applications 20

4.2 Edge-cloud Interworking & proposed algorithm 21

4.3 Simulation Results 25

4.4 Summary 27

CHAPTER 5: 28

DYNAMIC DECISION-MAKING ENGINE **28**
5.1 System model (Architecture components) 28
5.2 Proposed Algorithm 30
5.3 Evaluations 37
5.4 Summary 49

CHAPTER 6: **50**

CONCLUSION AND FUTURE SCOPE **50**
6.1 Conclusion 50
6.2 Future Scope 51

REFERENCES **52**

CHAPTER 1: INTRODUCTION

With the growing number of security concerns and easy access to cheap CCTV cameras, video analytics has become a norm across the world wherein right from a small shopkeeper in a remote village to sophisticated government organizations and big corporations are using multiple CCTV cameras for individual and public safety in their territory and analyzing video footage as per their capacity. In this chapter we discuss that while cloud computing options may seem favorable in some scenarios, edge computing provides an efficient and cost-effective approach to some hierarchical networks.

1.1 Cloud Computing

For a typical cloud computing scenario, data from multiple video sources, spread across a geographical location, needs to be processed for information gathering, and for this a centralized cloud server for performing video analytics applications such as facial recognition, object detection, tracking, etc are widely used.

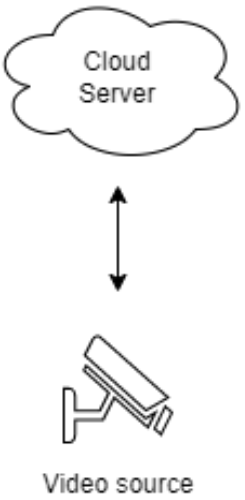


FIGURE 1.1: Typical cloud computing environment.

However, such an end-to-end setup has a lot of drawbacks.

1.1.1 Challenges in Cloud Computing

For a source to transmit an uninterrupted video stream with high resolution and framerate, it requires a network connection with high bandwidth and low latency, which in turn can attract a huge monetary charge. Failing to meet such network requirements can result in a jittery stream of low-resolution video frames, causing a loss of information. Very high latency between source and cloud can result in stale frames, defeating the objective of real-time processing. Moreover, the cost of operating a cloud service itself is very high, hence if the received frames are deemed useless for video analysis or from the perspective of the administrator, it can incur severe monetary losses.

Reducing such computational and monetary losses, especially in areas where these resources are very limited, is the main challenge we wish to encounter.

1.2 Edge Computing

The challenges faced in conventional cloud computing, as stated above, can be tackled by using edge servers. Edge servers are the fundamental block of a very famous distributed computing concept, known as Edge computing, that collects and embed data storage and computations closer to the data sources, reducing the proximity and transmission latency. The devices used as edge nodes are usually less powerful than cloud servers and less expensive to operate. The setup of an edge computing environment can be visualized in Fig 2. In such network architectures, the “big data” can be reserved for the cloud server owing to its superior computational capability, while the edge servers work on the “instant data” produced by the data sources.

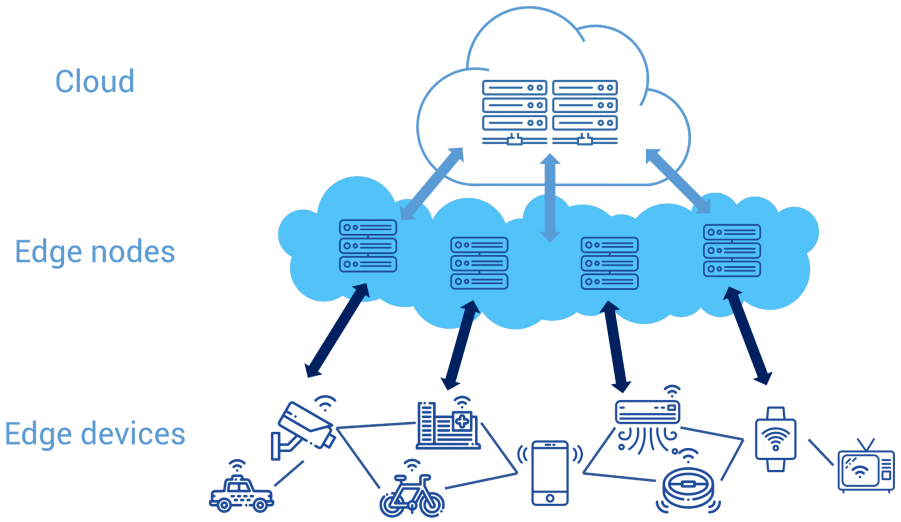


FIGURE 1.2: Setup of an edge computing environment.

1.2.1 Advantages of Edge Computing

Edge servers are more private and secure since the infrastructure reduces to a decentralized model and special encryption methods can be used as the data travels through the internet. Moreover, the ownership of the data collected shifts from the cloud service provider to the customers. The dynamic nature of such an infrastructure allows scaling as required and increases the reliability of the system as services can be accessed via other nodes in case a particular node fails. The proximity to the data sources conserves network resources and also increases the speed of operation, making the overall system very efficient.

1.3 The Need For Load Balancing

When a system is laid out in a vast area, network conditions of a particular geographical location can still be unfavorable and cause a hurdle in our video processing tasks. Performing video processing tasks at the edge nodes, Edge C3 i.e. caching, computing and communication challenges need to be taken into consideration. A certain location can have an excellent network, which will result in multiple video sources linked to its nearest edge server receiving a lot of video data with high resolution and high frame rates, increasing its computational load. The edge server is said to be in an over-utilized state. On the other hand, when network conditions are poor in a certain location, the video data accumulated at its

corresponding edge server is less and hence in an under-utilized state. This unbalanced computational “load” in the entire network reduces its efficiency.

In this thesis, an attempt has been made to provide a cost-effective algorithm for reallocating the computational overhead among multiple edge servers without the requirement of any modifications in the existing network layout. The algorithm is developed in an efficient manner such that it merely affects the execution time and the accuracy of the video processing tasks.

The computing tools that were used in implementation of this algorithm are mentioned below:

1. Programming Language: Python 3
2. Operating System: Microsoft Windows 10
3. Programming IDE: Visual Studio Code
4. RAM Capacity: 16GB

The next chapter, ie. Chapter 2 discusses the various literature surveys which are explained in brief about their problem statement, their applications, advantages and disadvantages, and how the proposed approach is superior to these algorithmic approaches.

Chapter 3 explains the DeepDecision algorithm approach which takes several network parameters and on the basis of these dynamic values, the edge server makes a decision for choosing the place of processing: either locally on the edge server itself or sending data to cloud server which will handle high computational workload.

Chapter 4 introduces another approach using an idea of Task Offloading-Job Balancing, where each video analytical application is considered a job, and smaller instances or parts of that video analytical application is coined as a task. Tasks are offloaded to the cloud servers, instead of the entire application, which was done in the case of DeepDecision algorithm. This helps in saving resources and time required to send the whole application to cloud, and latency plays a role whenever cloud computing is considered.

The proposed Dynamic Decision Making Algorithm is introduced in Chapter 5, which uses statistical calculations to decide the distribution pattern dynamically between the numerous edge servers processing videos from their front end video sources. Utilisation usage percentage is used as the comparison parameter for determining which edge servers are over utilised, and which are under utilised, and a balance between these extreme edge servers are computed and data is accordingly reallocated.

Chapter 6 concludes our research findings and results, and the future scope of the current proposed algorithm are stated.

CHAPTER 2: LITERATURE SURVEY

Numerous studies, conducted by researchers worldwide, were referred to in this study for the development of the proposed algorithm tackling the optimization problem in edge computing environments.

2.1 Edge Video Analytics for Public Safety: A Review.

Qingyang Zhang et al. published a survey article outlining the numerous applications and impacts of edge computing in our everyday lives. The article's main focus areas include police, transportation, firefighting, and EMS applications. The article also looks at the different video analytics methods and network topologies used in these applications.

2.2 Video Caching, Analytics, and Delivery at the Wireless Edge: A Survey and Future Directions.

The article presented by Behrouz Jedari et al. takes a comprehensive look into video edge-C3 challenges, i.e. caching, processing, and communication, as well as state-of-the-art solutions in next-generation wireless and mobile networks. In this article, more than 200 research contributions are discussed and compared, as well as insights into open issues and future research challenges, in an attempt to assist multimedia application developers and networking protocol designers in designing efficient video streaming and delivery solutions in upcoming wireless networks.

2.3 Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing.

Edge Intelligence (EI) was coined by Zhi Zho et al., who examined the impact of edge computing on artificial intelligence. This article provides a thorough discussion of the background and motivation for faster AI deployed in edge computing networks, as well as an overview of the broadscale frameworks, architectures, and developing essential technologies for deep learning model training/inference at the network edge.

2.4 Edge Computing Framework for Cooperative Video Processing in Multimedia IoT Systems

Due to limited network bandwidth, transmitting of large sized video packets over a long distance in multimedia Internet-of-Things (IoT) systems that integrate computer vision, and networking capabilities may cause congestion and delays in surveillance, automatic behaviour analysis, and event recognition. Changchun Long et al. proposed an edge computing framework to enable cooperative processing on resource-abundant mobile devices for time-sensitive tasks. A cooperative video processing scheme, composed of two efficient algorithms based on the derived optimal matching theorem has been proposed, which achieves suboptimal performance on human detection accuracy to tackle the above addressed problems. The suggested system has been tested with a variety of parameter settings as well as extensive simulations, which show that it outperforms the competition.

2.5 Computation Offloading Toward Edge Computing

Since the cloud is typically located far away from end devices, latency and quality of experience (QoE) for latency-sensitive apps suffer. Computations are outsourced to edge computing devices to achieve low latencies. This study by Li Lin et al. summarises current research on compute offloading in terms of application partitioning, job allocation, resource management, and distributed execution. Several disruptive application scenarios that are believed to be essential to the growth of edge computing are also illustrated in this paper.

2.6 DeepDecision: A Mobile Deep Learning Framework for Edge Video Analytics.

Deep learning methods are computationally costly, and front-end devices do not have enough CPU capacity for real-time processing. In this paper, Xukan Ran et al. offer a framework that connects front-end devices with computationally more powerful backend devices to enable deep learning algorithms to be deployed locally or remotely at the cloud/edge.

2.7 A Cost-Effective Deadline-Constrained Dynamic Scheduling Algorithm for Scientific Workflows in a Cloud Environment.

Novel ways that can leverage the advantages of workflow orchestration on cloud resources while also addressing the problems particular to a cloud environment must be developed in order to reap the true benefits of workflow orchestration on cloud resources. Jyoti Sahni et al. have proposed a dynamic, cost-effective, deadline-constrained heuristic approach for scheduling a scientific workflow in the cloud. The suggested method attempts to take advantage of the benefits of cloud computing while accounting for virtual machine (VM) performance variability and instance acquisition delay in order to establish a just-in-time plan for a deadline-driven scientific workflow at lower costs.

2.8 A Novel Edge-Cloud Interworking Framework in the Video Analytics of the Internet of Things.

Sanghong Ahn et al. introduce an unique edge-cloud interworking architecture for video analytics in interconnected systems that includes cost-effective task load balancing and scheduling techniques for video processing jobs demanding high compute power. The framework tries to reduce the cost of cloud resource utilisation while maintaining deadlines when performing concurrent processes. A two-stage mixed-integer issue is provided, along with its heuristic greedy algorithms, which captures all entangled goals.

2.9 Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications.

Ala Al Fuqaha et al. presented a survey on Internet of Things (IoT) with an objective of empowering technologies, protocols, and application issues, as well as an overview of the relationship between IoT and other evolving technologies such as big data analytics, cloud, and fog computing.

Studying the above articles helped us understand the importance of edge computing, their applications and the need for the optimization of the edge environment. The DeepDecision and the Task-Offloading-Job-Balancing algorithms were used as the basis for the formulation of our proposed algorithm. An attempt was made to replicate these algorithms in a simulated environment, which are discussed in the next chapters, to understand the dynamic behaviour and other aspects of the various parameters involved. The parameters of concern were narrowed down and made the primary focus in our optimization challenge.

We intend to achieve the following objectives through the DDME algorithm:

1. Network topology and pipeline of video analytics applications simplified.
2. Caching, Computing and Communication challenges, the Edge C-3 balanced.
3. Faster AI employment in edge computing networks.
4. Cooperative processing on resource-abundant edge devices.
5. Improving Quality of Experience (QoE).
6. Real time processing and efficient resource management.
7. Deadline-driven scientific workflow.
8. Reducing cost of cloud resources.

CHAPTER 3: PRELIMINARY WORK

In this chapter we have discussed in detail the algorithms developed by researchers, noted in the literature survey. In our preliminary testing, an attempt was made to replicate these algorithms which allowed us to benchmark them against our proposed algorithm and inspect the pros and cons of each.

3.1 The DeepDecision Algorithm

Xukan Ran et al. proposed an algorithm, in their paper titled “DeepDecision: A Mobile Deep Learning Framework for Edge Video Analytics”, which takes into consideration the various network and video parameters in an edge computing framework and decides “where and which” deep learning model should be run based on the application requirements such as accuracy, frame rate, energy, and network data usage.

The experimental setup in their study consists of front-end android devices, capable of running two variants of the YOLO CNN deep learning model for object detection: TinyYOLO with 9 convolutional layers and BigYOLO with 22 convolutional layers, designed using Tensorflow for Android. The difference between the two deep learning models lies in terms of accuracy and speed, the BigYOLO being more accurate but slow and vice-versa for TinyYOLO. The front-end devices are connected to edge servers or “backend-helpers”, which is essentially a computer system equipped with an Nvidia GPU, running only the BigYOLO CNN.

The basic working principle of the DeepDecision algorithm can be visualized in Fig 3.1.

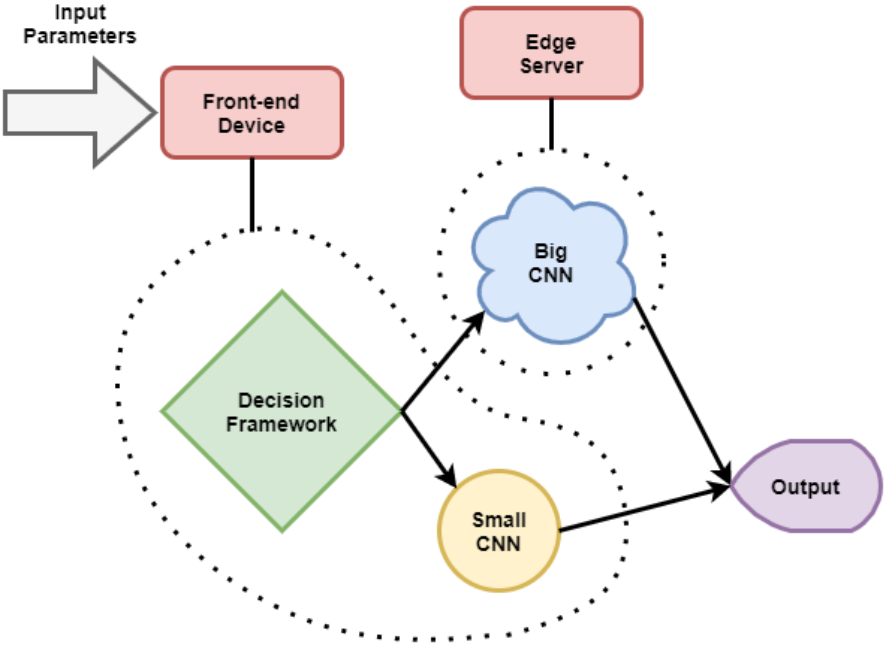


FIGURE 3.1: Working principle of the DeepDecision algorithm.

The approach is a data-driven optimization framework that takes as input the empirical measurements of these tradeoffs, and computes the optimal combination of decision variables that maximizes the key metrics, and outputs the optimal decision. The framework is designed to be general enough to handle any values of input measurement data while still capturing the tradeoffs between decision variables and metrics. The list of input parameters considered in this algorithm is listed in Table 3.1.

TABLE 3.1: Table of Notations. $i = 0$ represents remote execution, and $i = 1, \ldots, N$ represents local execution on the front-end device.

Variable	Description
p	frame resolution (pixels^2)
r	video bitrate (bits/s)
f	frame rate (frames/s)
$y_i(t)$	model decision at time t
$a_i(p, r, \ell_i)$	accuracy of model i (%)
$b_i(p, r, f)$	battery of model i (J/s if $i = 0$, J/frame if $i > 0$)
$\ell_i(p, r, f)$	total delay when using model i (s/frame)
$\ell_i^{\mathrm{CNN}}(p)$	processing delay from running model i
B	network bandwidth (kbps)
L	network latency (s)
$\mathcal{B}$	target battery usage (J/s)
A	accuracy target (%)
F	frame rate target (frames/s)
c	monetary cost ($/bit, if use cellular network)
C	target monetary cost ($/bit)
α	parameter that trades off accuracy for frame rate in the objective function

There are certain constraints bounding each network parameter that has to be taken into account, and accordingly, an optimization problem was to be solved at the beginning of each interval to decide the deep learning algorithm's configurations.

The objective is to maximize the number of frames sampled plus the accuracy of object detection in each frame. The constraints bound while developing the DeepDecision algorithm were as follows:

1. The total delay experienced by a frame is equal to the CNN's processing time plus the network transmission time (if applicable).
2. The frame rate cannot be chosen to exceed the processing time of the CNN (remote or local).
3. The video cannot be uploaded at a higher bit rate than the available bandwidth.
4. The battery usage cannot exceed the maximum target.
5. The monetary cost cannot exceed the maximum target.

The DeepDecision algorithm is described mathematically in Fig 3.2.

Input: Target cost C, target battery $\mathcal{B}$, cost per bit c, network bandwidth B, network latency L, model battery usage function b_i, model latency function ℓ_i, model accuracy function a_i
Output: Frame resolution p^*, video bitrate r^*, frame rate f^*, decision of model variant $\mathbf{y}^*$

$f \leftarrow \frac{1}{f_0^{\text{CNN}}}$ ▷ remote model
$p, r \leftarrow \arg\max_{p,r \leq \min(B, b_0^{-1}(\mathcal{B}|B), \frac{C}{c})} (f + a_0(p, r, \ell_0^{\text{CNN}} + \frac{r}{fB} + L))$
if $a_0(p, r, f) \geq A$ **and** $f \geq F$ **then**
    $u_{\max} \leftarrow f^* a_0(p, r, f)$
    $r^* \leftarrow r, f^* \leftarrow f, p^* \leftarrow p, \mathbf{y}^* \leftarrow e_i$
for $i \leftarrow 1$ to N **do** ▷ try the local models
    $r \leftarrow r_{\max}$ ▷ don't need to compress locally
    $p \leftarrow \arg\max_p (\min(\frac{1}{\ell(p)}, \frac{\mathcal{B}}{b(p)}) + a_i(p, \ell_i(p))$
    $f \leftarrow \min(\frac{1}{\ell(p)}, \frac{\mathcal{B}}{b(p)})$
    $u \leftarrow f + a_i(p, \ell_i(p))$
    if $u > u_{\max}$ **and** $a_i(p) \geq A$ **and** $f \geq F$ **then**
        $u_{\max} \leftarrow u$
        $r^* \leftarrow r, f^* \leftarrow f, p^* \leftarrow p, \mathbf{y}^* \leftarrow e_i$
return $p^*, r^*, f^*, \mathbf{y}^*$

FIGURE 3.2: Mathematical description of the DeepDecision algorithm.

3.2 Test Setup Used for Replicating DeepDecision

For benchmarking any algorithm, the test simulation environment must be as close as possible to the proposed methodology in the original research. An attempt was made to accomplish the same with the resources available at hand during this study.

3.2.1 Hardware Setup

Two computer systems were used for replicating the experimental setup in the original research conducted for DeepDecision. The computationally less powerful system was labeled as the front-end or "client", and the other as the backend or "server".

The DeepDecision algorithm was replicated in Python 3.8 and executed on the client device. The various network and device parameters were calculated in real-time using various libraries available in Python such as speedtest, socket, etc.

3.2.2 Connection between the client and server

To mimic the environment of an edge computing setup, the video frames produced by the client were transmitted over the internet using the NGROK.

NGROK is a freemium application that exposes a local port using an encrypted TCP tunnel that provides an internet-accessible address. Anyone with the said link can access the data being transmitted over the localhost. All the requests made over the HTTP connection are logged on the screen as shown in Fig 3.3. This application was run on the client and the link was registered on the server. The decision to transmit the frames is made by the DeepDecision algorithm.

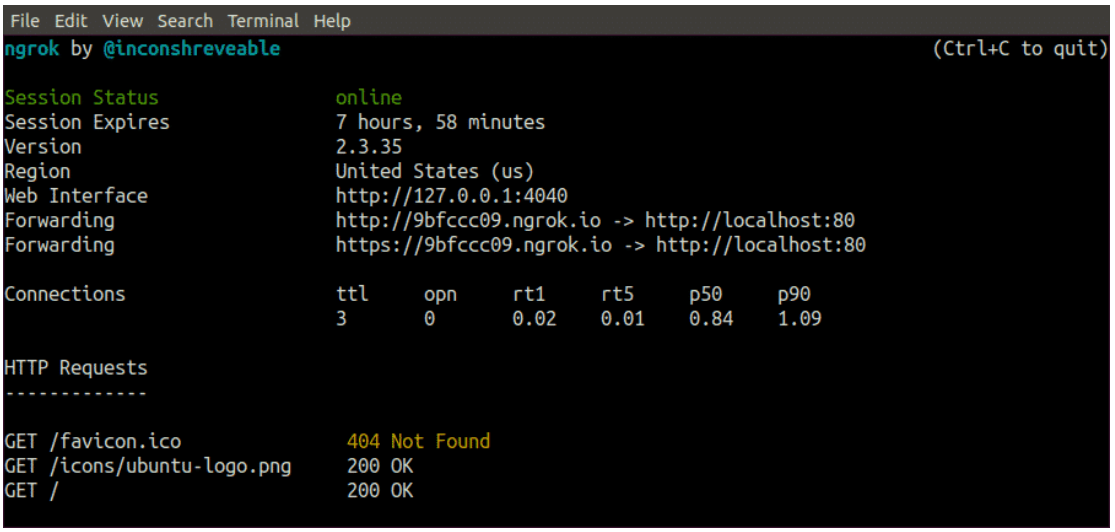


FIGURE 3.3: NGROK window post successful connection between client and server.

The premium version of NGROK allows for generating a fixed URL for establishing a connection between more than two devices.

3.2.3 Deep Learning Models

YOLO CNN is a famous computer vision algorithm used for real-time object detection in an image or video frame. It requires only one forward propagation

through the neural network to detect the objects. The network is trained on the COCO dataset which consists of 328,000 images of 80 classes of various objects such as vehicles, animals, daily use objects, etc. Bounding boxes are generated around the objects present in the input image as shown in Fig 3.4.

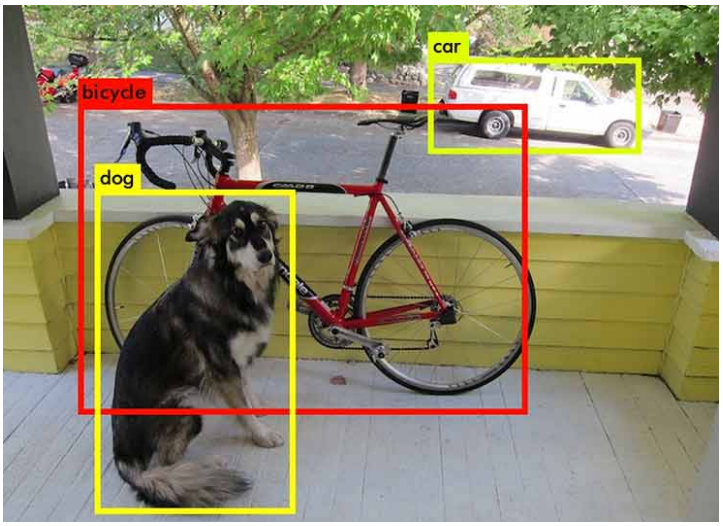


FIGURE 3.4: Sample output of object detection using YOLO CNN.

Over the years, many variants of the YOLO architecture naturally, ed, every version better than the previous in terms of accuracy and detection speed. The third version of the TinyYOLO and BigYOLO has been used for this study. The performance comparison has been tabulated in Table 3.2.

TABLE 3.2: Performance comparison of TinyYOLOv3 vs BigYOLOv3.

YOLO Model	**mAP**	**FLOPS (in Billions)**	**FPS**
TinyYOLO v3	33.1	5.56	220
BigYOLO v3	60.1	141.45	20

The architecture and weights files were imported from the publicly available Darknet repository and deployed into our testing program using the OpenCV library's DNN function.

Based on the decision made by the DeepDecision algorithm, the corresponding object detection process is executed. For example, if the network

bandwidth and the processing capacity of the front-end device are not sufficient, the TinyYOLO architecture on the front-end device itself is used to make the best of the situation. Likewise, if the network conditions are excellent and the accuracy of detection is prioritized over real-time processing, the data is transmitted to the server and the BigYOLO architecture is used.

The video frames transmitted had a variable resolution ranging from 160×160 to 480×480 pixels. The YOLO architecture can dynamically adjust its internal structure for different resolutions, so its execution time is sensitive to the video resolution.

3.3 Simulation Results

With the various parameters being handled by the DeepDecision algorithm, it is essential to observe how their variation affects the decision-making process and eventually the performance of YOLO CNN models.

Naturally, higher resolution frames contain more information and their extraction will require more time. This trend can be observed in Fig 3.5. Since the TinyYOLO algorithm performs less number of computations on the input frame, the processing time is less than that of BigYOLO.

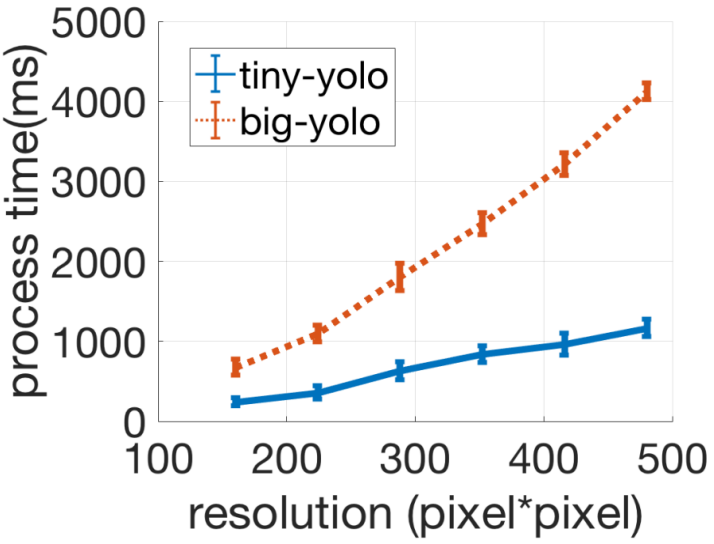


FIGURE 3.5: Processing time required for various video frame resolutions.

Consequently, the longer the deep learning model takes to perform its computations on a certain input, the more energy consumption of the corresponding device. This can be observed in Fig 3.6.

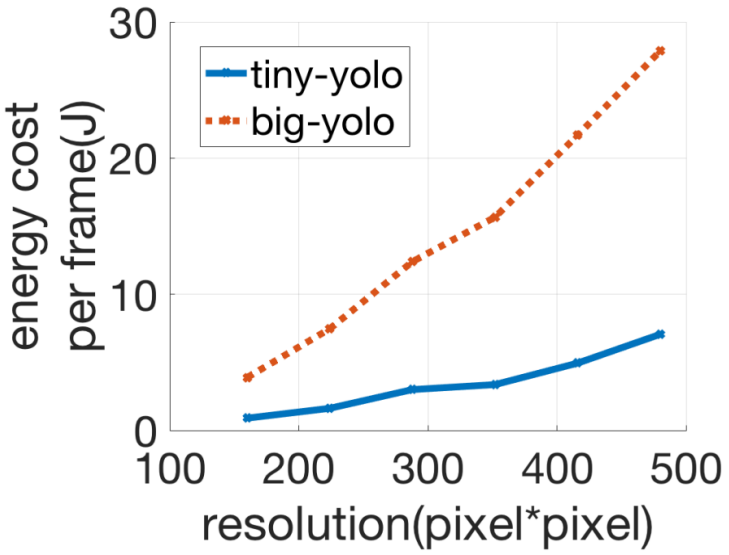


FIGURE 3.6: Energy consumed per frame by the device for various video frame resolutions.

The accuracy of the YOLO CNN is measured using the IoU method formulated by:

$$IoU = \frac{area(R \cap P)}{area(R \cup P)}$$

where R and P are the bounding boxes of the ground truth and the model under evaluation, respectively. This accuracy varies with the input resolution and bitrate of the video which can be observed in Fig 3.7. Higher resolutions and bitrates mean more information is available per frame which can be extracted.

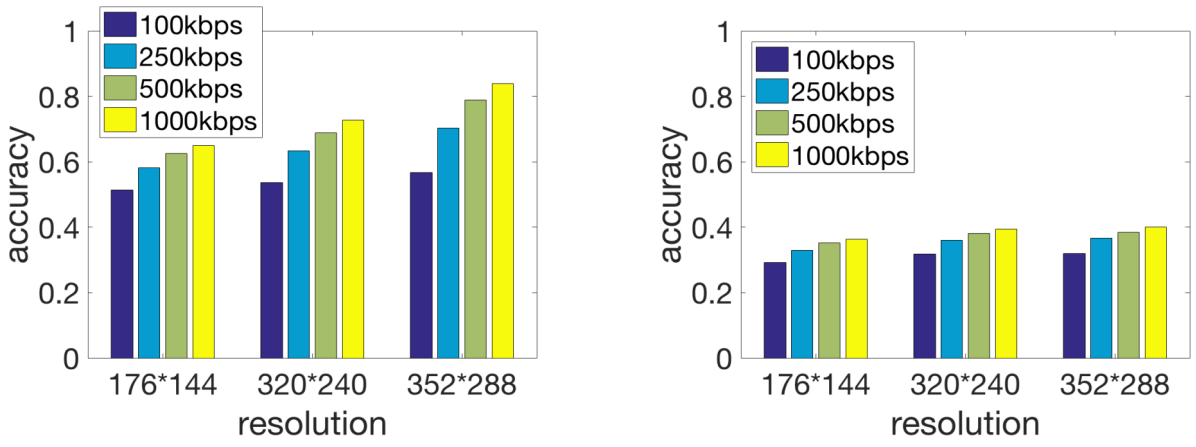


FIGURE 3.7: Variation of the accuracy of YOLO CNNs with input resolution and bitrates.

When the video frames are offloaded to the edge server, the effects of network latency and bandwidth come into play. As stated earlier, the server runs only BigYOLO CNN. On a particular input resolution, the processing time remains more or less constant. The network latency tends to reduce as the bandwidth increases, although this case may not always be necessary; a certain connection can have excellent bandwidth but terrible latency. The trend of network latency paired with the processing time at the server paired can be seen in Fig 3.8. The minor deviation in processing times has also been accounted for.

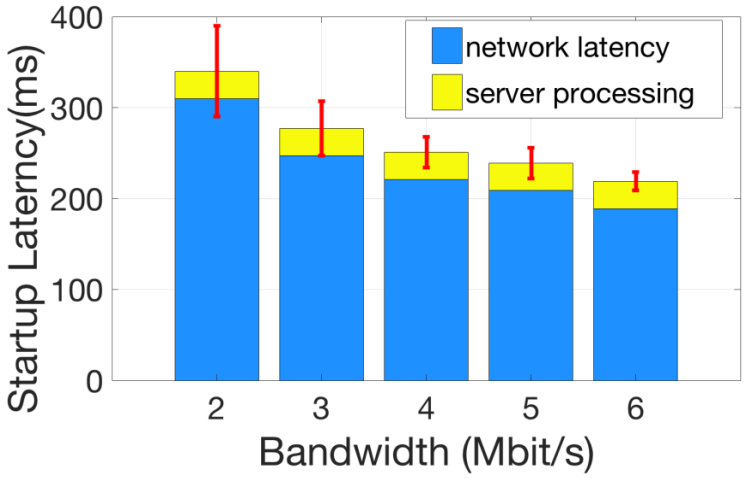


FIGURE 3.8: Trends in total system latency vs network bandwidth.

From the above results, it is evident that the design of the TinyYOLO architecture is aimed at improving the speed of the model rather than giving accurate results whereas these design goals are inverted for the BigYOLO architecture. The performance of these deep learning models varies with the network and input video parameters of the network layout.

3.4 Summary

In this chapter, we have discussed the DeepDecision algorithm proposed by Xukan Ran et al. which dynamically chooses the location for the execution of the computations occurring in an edge computing environment based on the network

conditions like latency and bandwidth, and various input video parameters like resolution, framerate, and bitrate. The algorithm also optimizes the parameters of the input video post, deciding to deliver a real-time performance from the deep learning model.

The algorithm was successfully replicated with a similar experimental setup as proposed in the original research paper. The connection between the client and server devices was established using the NGROK application.

We also discussed object detection using YOLO CNN, compared the performance of its variants, namely TinyYOLOv3 and BigYOLOv3 and used it as a benchmarking tool for this study. The effects of the various parameters considered in this research experiment were measured extensively, visualized using graphs, and recorded for further comparison against the algorithm proposed by us in this thesis, which will be discussed in the next chapters.

CHAPTER 4: THE TASK OFFLOADING-JOB BALANCING ALGORITHM

The Task Offloading-Job Balancing algorithm approach uses cost-effective scheduling schemes for load distribution for processing of various stages in a video analytics application.

4.1 Video analytics & its applications

Image processing is being highly used in general public service departments, and each department has specific jobs, and these are illustrated below in Fig 4.1:

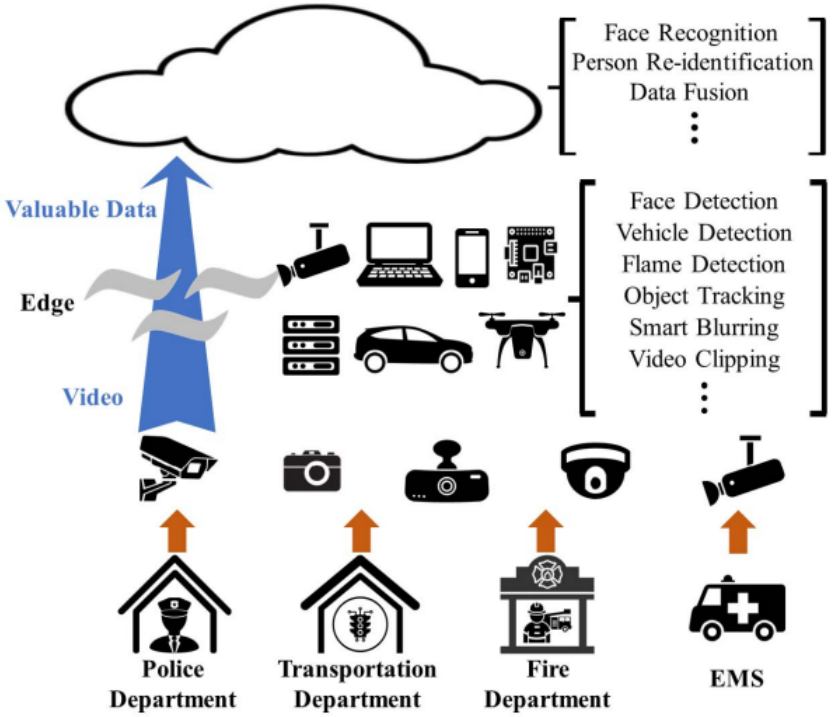


FIGURE 4.1 Various applications of video analytics.

We observe that jobs vary from: Face Recognition, Vehicle Detection, Flame Detection, Object Detection, etc. Let us consider any job to understand sub-jobs inside them, for example, Object Detection.

A typical process of a video analytics application is shown below:

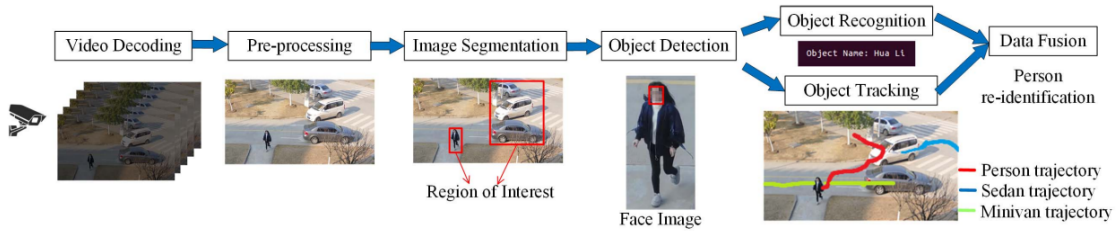


FIGURE 4.2: Pipeline of a video analytics application

4.2 Edge-cloud Interworking & proposed algorithm

For different requirements of our application (ie. “jobs”), we divide the entire processing into small “tasks”, where each task is as seen above: Video Decoding, Pre-Processing, Image Segmentation, Object Detection, Object Recognition and so on. This algorithm makes use of an edge-cloud interworking architecture, to incorporate the division of tasks and jobs.

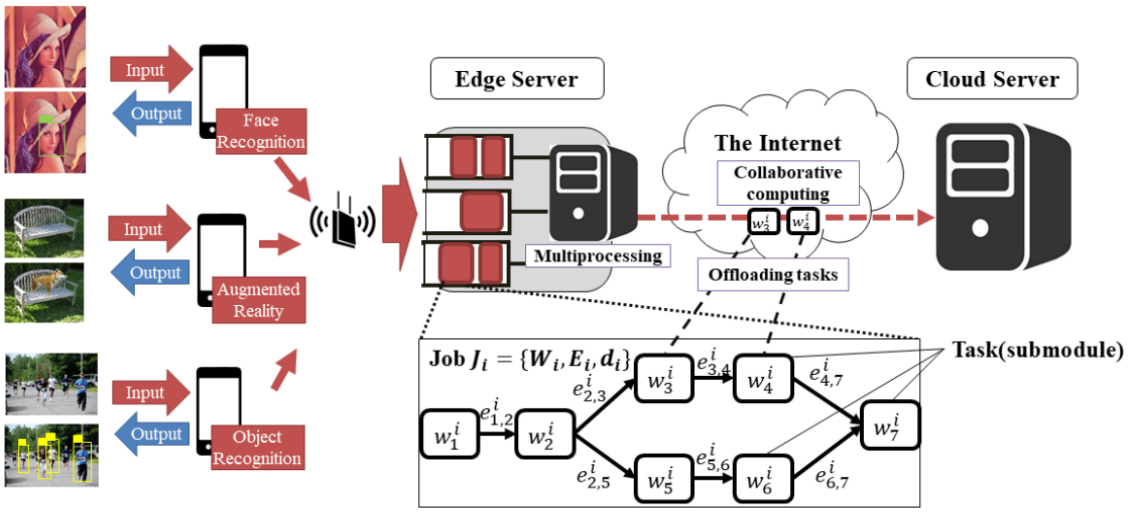


FIGURE 4.3 Edge-cloud interworking mechanism.

In this architecture, heavy computational tasks, such as object detection or object recognition, are offloaded to the cloud server, as the edge infrastructure is not capable of processing these in the given deadlines, to ensure real-time processing.

The decision to make in this case is between sending the tasks and not the entire job (which was done in the DeepDecision algorithm in the previous chapter).

This ensures that some tasks are processed on the edge and simultaneously heavier tasks are done by the cloud, and this ensures saving cost of resources and time.

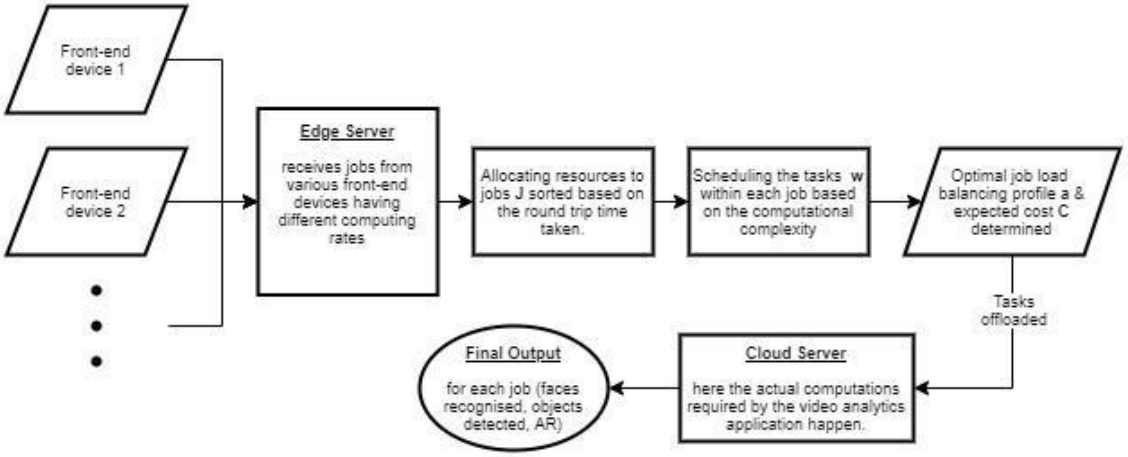


FIGURE 4.4: The processing flow for the task offloading-job balancing algorithm.

The implementation of this architecture has been divided into 2 algorithms, described mathematically in Fig 4.5 and Fig 4.6.

Algorithm 1 Greedy Task Offload Scheduling Algorithm

Input: Profile of i_{th} job $J_i = \{\boldsymbol{W}_i, \boldsymbol{E}_i, d_i\}$
Input: Assigned computing rate f_i
Output: Optimal offload schedule profile $\boldsymbol{a}_i$
Output: Optimal expected fee C_i

procedure SCHEDULE(J_i, f_i)
  $\bar{t}_i \leftarrow \sum_{j=1}^{|\boldsymbol{W}_i|} \left(t_{i,j}^{\text{cloud}} - t_{i,j}^{\text{edge}} \right)$
  $C_i \leftarrow \sum_{j=1}^{|\boldsymbol{W}_i|} c_j^i$
  $List \leftarrow$ Sorted list of $\boldsymbol{W}_i$ by $q_j^i(X_j^i), \forall j \in [0, |\boldsymbol{W}_i|]$ in descending order.
  $limit \leftarrow \bar{t}_i$
  for $j \leftarrow 1$ **to** $|\boldsymbol{W}_i|$ **do**
    $Index \leftarrow List[j]$'s original index
    $temp \leftarrow limit - (t_{i,Index}^{\text{edge}} + t_{i,Index}^{\text{transfer}})$
    if $\boldsymbol{a}[k] == 0$ for $k \in P_{Index}$ **then**
      $temp \leftarrow temp + t_{i,Index}^{\text{transfer}}$
    end if
    if $temp \geq 0$ **then**
      $\boldsymbol{a}[Index] \leftarrow 0$
      $limit \leftarrow temp$
      $C_i \leftarrow C_i - c_{Index}^i$
    end if
  end for
  return $\boldsymbol{a}$, C_i
end procedure

FIGURE 4.5: Greedy Task Offload Scheduling Algorithm.

The Task Offload Scheduling algorithm has the objective to minimize the cost model, but also ensure that deadline constraints are met.

The Job Load Balancing algorithm has the objective to find the computing rates for concurrent jobs under consideration, which not only minimizes the total monetary cost but also performs this in the given deadlines.

Algorithm 2 Greedy Job Load Balancing Algorithm

Input: List of job profiles $J = \{J_1, \ldots, J_N\}$
Output: Optimal job load balancing profile $\boldsymbol{f}$
Output: Optimal expected fee C

procedure RESOURCEALLOCATION(J)
  Let $r_i = \sum_{j=1}^{|\boldsymbol{W}_i|} q_j^i(X_j^i)/d_i, \forall i \in [1, N]$.
  $List \leftarrow$ Sorted list of jobs $\in J$ by r_i in descending order.
  $limit \leftarrow f^{\text{edge}}$
  for $i \leftarrow 1$ **to** N **do**
    $Index \leftarrow List[i]$'s original index
    $temp \leftarrow limit - r_{index}$
    if $temp \geq 0$ **then**
      $limit \leftarrow temp$
      $f_{Index} \leftarrow r_{Index}$
    else
      break
    end if
    $(\boldsymbol{a}_{Index}, C_{Index}) \leftarrow schedule(J_{Index}, f_{Index})$
  end for
  if $limit > 0$ **then**
    $j \leftarrow \arg\min_k C_k(\boldsymbol{a}_k^*)$ where $k \in \{i, \ldots N\}$ and $f_k \leftarrow limit$
    $Index \leftarrow List[j]$'s original index
    $f_{Index} \leftarrow limit$
    $(\boldsymbol{a}_{Index}, C_{Index}) \leftarrow schedule(J_{Index}, f_{Index})$
  end if
  return $\boldsymbol{f} \leftarrow \{f_1, \ldots, f_N\}$, $C \leftarrow \sum_{i=1}^{N} C_i$
end procedure

FIGURE 4.6: Greedy Job Load Balancing algorithm.

In the Greedy Task Offload Scheduling Algorithm, we follow these steps:

1. Sort jobs by the amount of computing required in descending order. *List* is the name for this ordered list.
2. Initialize a queue for branch-first search and a variable called *max_val*, which represents the largest value in the search. The queue is filled with a fake node with a value of 0 and a level of 0. For all jobs to be offloaded, an initial offloading profile is defined. ($a_j^i = 1$).

3. Repeat the process until the queue is no longer full:
    a. Dequeue a task and give it the name *u*. *v* is the job in *List* at the next level of *u*.
    b. Calculate the service latency and cost savings when *v* is processed at the edge ($a_v^i = 0$). If the service delay under the current profile is less than the deadline and the value is greater than *max_val*, update the value of *max_val* to the value and save the current profile..
    c. When *v* is processed at the edge, calculate the bound. By applying the greedy solution of the LP approximation to the following elements of *v* in *List*, we may get the bound.
    d. Put *v* in the queue if the bound is greater than or equal to *max_val*.
    e. For the scenario of ($a_v^i = 1$), process 3-b through 3-d, which indicates that *v* will be offloaded to the cloud.

In the Greedy Job Load Balancing algorithm, we follow these steps:

1. Sort jobs in descending order based on their needed compute rates.
2. Assign the necessary compute rate to the job starting with the first one.
3. If Step 2 isn't achievable, choose a job that allows the edge to do as many tasks as possible with the remaining processing capacity. Assign the remaining computing resources to that task.

The evaluation parameters used in the above algorithms are tabulated in Table 4.1.

TABLE 4.1: Evaluation parameters for the above algorithms.

Parameter	Value
Cloud server's computing capability	17.6 GFLOP/s [5]
Edge server's computing capability	4.4 GFLOP/s [5]
Transferred data size between tasks	[250, 500] KB
Average round-trip time, r^{cloud}	80 ms
Number of concurrent jobs, N	4
Job deadline	[300, 600] ms
Task flow dependency $\boldsymbol{E}_i$	$\{e^i_{j-1,j}, \forall j\}$
Computation complexity $q^i_j(X^i_j)$	$7 \cdot exp(x^i_{j-1,j}, 1.5)$, where $X^i_j = \{x^i_{j-1,j}\}$ [3]
Cloud usage fee, b^{cloud}	$ 1 per GFLOP

4.3 Simulation Results

We replicated this algorithm using Python 3.8 and evaluated this proposed framework on the basis of a Monte Carlo simulation of 1000 random cases. The performance evaluations are illustrated figures are given below:

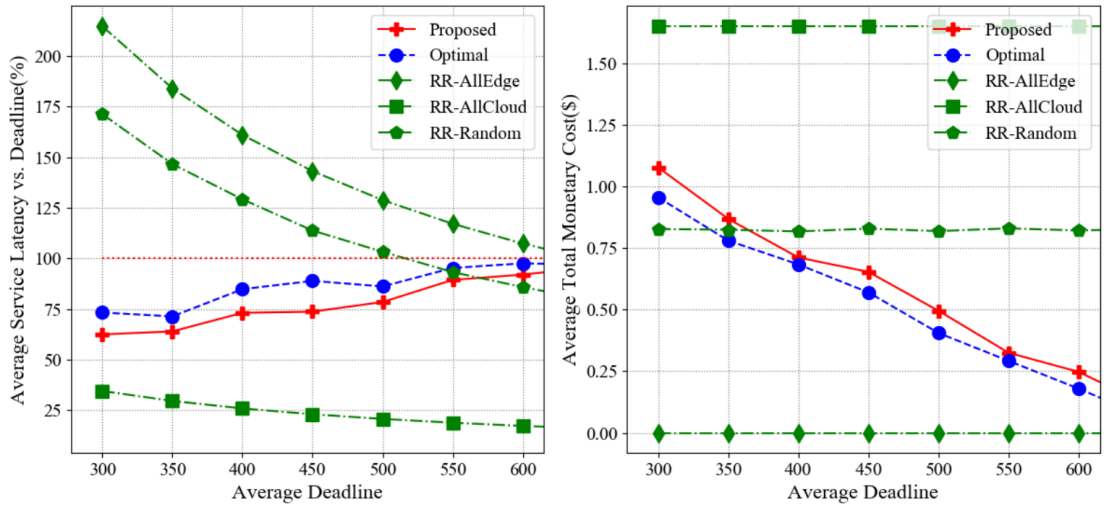


FIGURE 4.7: Average service latency and average total monetary cost w.r.t. deadline.

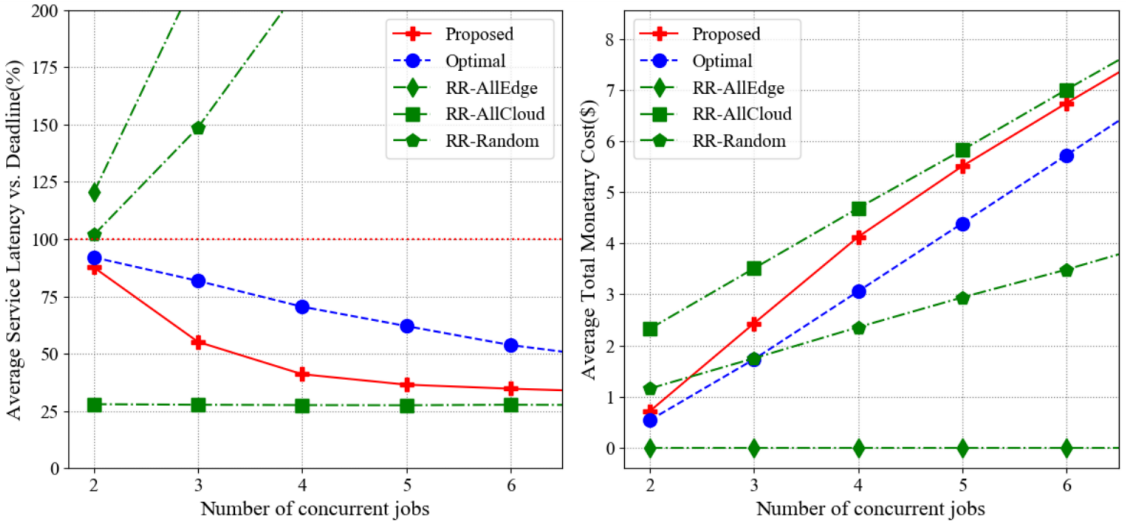


FIGURE 4.8: Average service latency and average total monetary cost w.r.t. the number of concurrent jobs.

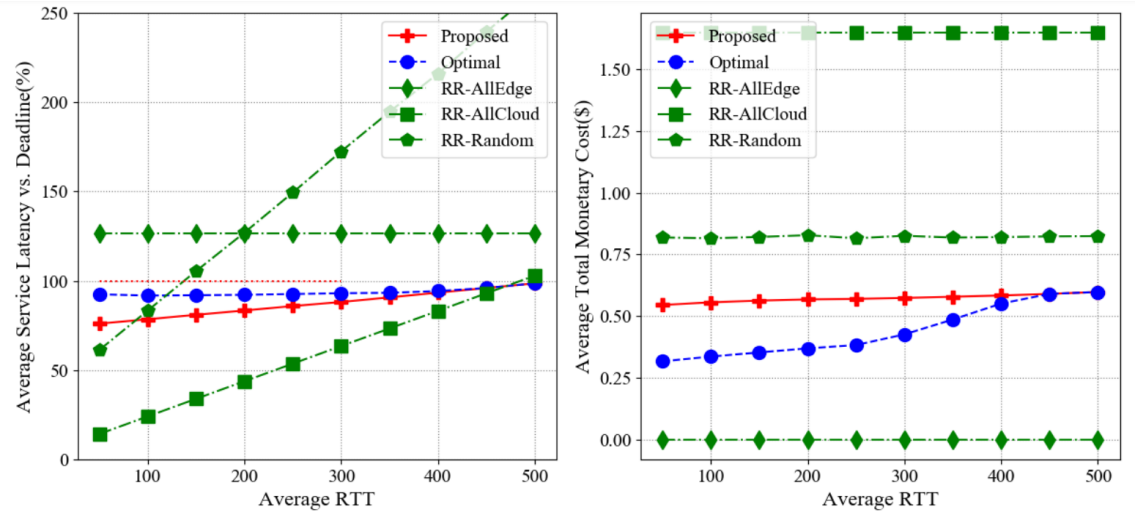


FIGURE 4.9: Average service latency and average total monetary cost w.r.t. average RTT (round trip time).

The notations in the label refer to

1. RR-AllEdge, which stands for round-robin (RR) workload balancing without offloading the task to the cloud server.
2. RR job load balancing with full task offloading to the cloud server (RR-AllCloud).
3. RR-Random workload balancing using random task offloading to the cloud server.
4. Optimal is a metaheuristic algorithm for workload balancing using BnB task offload scheduling that uses a harmonic search technique.

Figure 2 depicts the average service delay and overall monetary costs of multiple systems with varying average deadlines (300 ms to 600 ms for a task). It's worth noting that the y-axis' average service latency shows the ratio of completed service latency to the required deadline.

Even though it does not require monetary expenditures, RR-AllEdge fails to meet the deadline criteria of jobs with a deadline of less than 400 milliseconds. This is because the edge server's computational capabilities are insufficient to process several jobs in such a short period of time. RR-AllCloud, on the other hand, outperforms the others in terms of average service latency while also having the highest financial costs. RR-Random has lower financial costs than RR-AllCloud, but it still has more delay than the average service. Finally, the proposed system achieves entangled goals,

such as lowering monetary costs while meeting time constraints, by considering both service latency and monetary costs.

When the number of concurrent jobs is varied from two to six, the results of many systems are shown in Figure 3. As shown in Fig. 2, RR-AllEdge and RR-Random do not provide deadline limits as the number of concurrent jobs grows. RR-AllCloud also has the highest financial outlay, although having a little advantage in terms of average service latency.

Finally, findings with average RTTs r^{cloud} ranging from 50 to 500 ms are shown in Fig. 4. As the average RTT (i.e. transmission latency) rises, the suggested technique looks to be conservative when shifting work to the cloud. This is because the proposed approach tries to decrease the number of execution site changes, which causes transmission delays. The average service latency of the proposed scheme has a lower bound of 450 ms, which is achieved by RR-AllEdge. Despite this, the high-baseline transmission latency prevents it from meeting the deadline criteria [300, 600 ms].

4.4 Summary

In this chapter, we have discussed the Task Offloading-Job Balancing algorithm proposed by Sanghong Ahn et al. which proposed an edge-cloud interworking architecture, and each analytical application was denoted as a “job”, and each subpart of the application was denoted “task”.

The algorithm was successfully replicated with a similar experimental setup as proposed in the original research paper. A job of 8 tasks that were connected in a row was considered, and the sizes of passed data between the tasks are randomly determined in a range of [250, 500] KB.

We also discussed the evaluations of the algorithms which were based on a Monte Carlo simulation and compared the performance for different deadline constraints ranging between [300, 600]ms. The effects of the various parameters considered in this research experiment were measured extensively, visualized using graphs, and recorded for further comparison against the algorithm proposed by us in this thesis, which will be discussed in the next chapter.

CHAPTER 5:
DYNAMIC DECISION-MAKING ENGINE

In chapters 3 and 4 we discussed in detail two proposed frameworks, DeepDecision and Job Balancing-Task Offloading algorithms, which aim to optimize the computational overhead in an edge computing environment. However, their proposed algorithms consider working with only one edge server in their network layout. In practical applications such as monitoring the traffic of an entire city, there may be several edge servers that are placed in the vicinity of video sources to reduce computational overhead at cloud servers on the Our Dynamic Decision-Making Engine (DDME) framework intends to generalize the optimization objective to a vast network layout rather than focusing on a small region.

5.1 System model (Architecture components)

When video sources are spread out far apart geographically, network conditions are very dynamic for each edge server. This causes certain edge nodes to be in an over-utilized or under-utilized state depending on the amount of data it receives from the various video sources under its umbrella. Our DDME framework adds to the existing network layout to balance the load on resources of all edge servers to bring them back to an optimum utilization state.

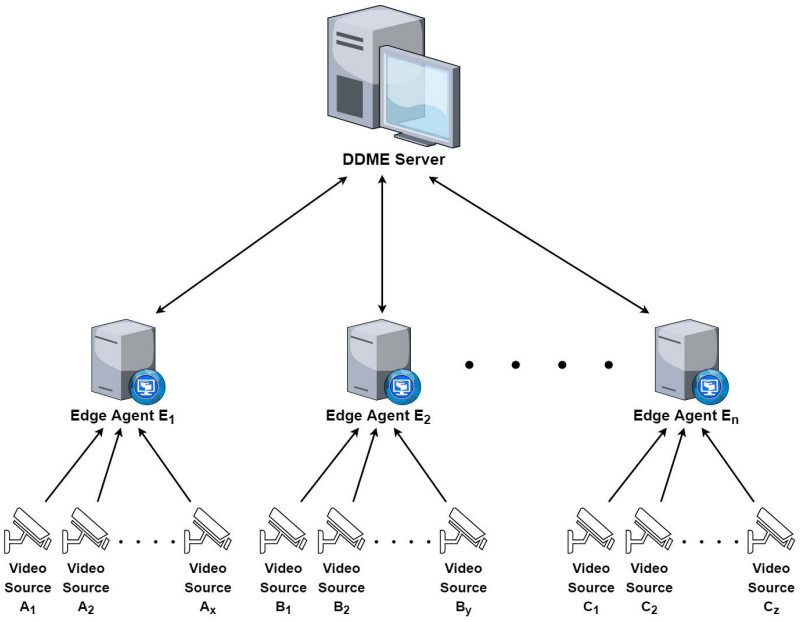


FIGURE 5.1 System architecture with DDME server handling a variable number of edge agents and video sources.

DDME server is set up between the edge-cloud pathway, to minimize the need to transfer data to the cloud for faster processing. This setup requires little to no modification in the existing infrastructure hence incurring no additional monetary expenses. The system layout taken under consideration is depicted in Fig 5.1.

Video sources such as CCTV cameras are installed throughout the city. An edge server is assigned to a group of geographically close sources, and all the edge servers are connected to a centralized cloud server. Various algorithms were used by different researchers for collaborative load distribution among edge servers [3–7]. However, different locations tend to exhibit different network conditions such as bandwidth and latency, consequently affecting the parameters of the video stream. Higher bandwidth allows streaming at higher resolutions and bitrates, while lower latencies result in higher frame rates. The ideal combination of such parameters can result in a large data buffer transferred to the edge device which in turn increases the processing overhead. The size of the data buffer can be estimated using the formula 1 and Table 1.

$$Size\ (in\ bits) = Resolution \times Frame\ rate \times Bits\ per\ pixel \times Video\ duration \quad (1)$$

TABLE 5.1: Video frame resolution vs required streaming bandwidth & data buffer size per frame.

Frame Resolution	Required Bandwidth (Mbps)	Size of data buffer per frame (in Bytes)
144p	0.1	460.8
240p	0.4	1278
360p	0.7	2880
480p	1.1	5124
720p	2.5	11520
1080p	5	25920
1440p	12	46080

5.2 Proposed Algorithm

In our proposed framework, we will be dealing with two algorithms i.e.

1. Data Reallocation Algorithm.
2. Dynamic Decision-Making Engine (DDME) Algorithm.

Every 5 seconds, the DDME server and edge agents have to perform the following algorithms:

A) Algorithm 1: Data Reallocation Algorithm

1. Each Edge Agent receives video frames from their respective multiple video sources. Each video frame from a video source with a respective resolution occupies sizes corresponding to Table 1.
2. Total space of an edge agent is calculated by the total size of video frames from all its respective video sources. This total size is updated every 5 seconds.
3. Threshold value for each edge agent is calculated dynamically as per the edge agent space restrictions due to other processes running in the background. For example: for a 2GB total space, if 1.75GB is occupied due to background process, then available space is 250MB which is hence set as the threshold, and if the total space of an edge agent exceeds 80% of 250MB (ie.200MB) or is less than 40% of 250MB (ie. 100MB), the edge agent sends a signal to DDME server to reallocate data in these edge agents.
4. This threshold calculation is done for other edge agents similarly and this threshold value is determined dynamically hence will be different at every iteration.
5. If the total space of an edge agent is between 40% and 80% (ie. 100MB<total size<200MB), no reallocation is required.
6. After DDME Server returns reallocation information, frames from respective edge agents are transferred to/received at their paired edge agent.
7. Repeat steps 2 to 5

B) Algorithm 2: Dynamic Decision Making Engine Algorithm

1. Receive List of Agents with their respective total data size percentage.
2. Classify them into two buckets, 1st bucket where the data size of the edge server is more than 80% of the threshold value, and the second bucket where the data size is less than 40% of the threshold value.
3. Sort them in their respective buckets (Descending in more than 80% bucket and ascending in less than 40% bucket). Based on sorted values, pair up edge servers.
4. Send back a list with agents having the new total size in the range of 40%-80%, and send a notification to the edge server with more consumption to send frames to the corresponding paired edge server which has more capacity.

Table 5.2 contains notations for all variables used in algorithms.

TABLE 5.2: Table of Notations.

Variable	Description
$J = \{J_1, J_2, ..., J_N\}$, where i_{th} agent $J_i = \{E_i, \boldsymbol{V}_i, d_i\}$	List of agents, with information about edge_id, video sources handled by that edge, and the total data size at that edge agent.
E_i	Edge_id of each edge agent, used to track videos sent to/from which agent.
V_i	List of Videos from the respective edge agent's video sources
d_i	Total data size at the edge agent which is used for calculation of Space%.
A	List of Space% of agents having Space%>80% or Space%<40%
$\bar{A}$	List of Space% of agents having 40%<Space%<80%
A'	Updated list of Space% of agents having Space%>80% or Space%<40%
List_80	Sorted list of elements of A having Space%>80%, in descending order.
List_40	Sorted list of elements of A having Space%<40%, in increasing order.
$J' = \{J'_1, J'_2, ..., J'_N\}$, where i_{th} agent $J'_i = \{E_i, \boldsymbol{V}'_i, d'_i\}$	List of agents, with information about edge_id which remains same, updated video sources handled by that edge, and the new total data size at that edge agent.

Algorithm 1 Data Reallocation Algorithm

Input: List of Agents $J = \{J_1, J_2, ..., J_N\}$ with their respective videos and data sizes, where i_{th} agent $J_i = \{E_i, \boldsymbol{V}_i, d_i\}$

Output: Updated list of Agents J', with optimised data sizes and reallocated video-frames

procedure DATA_ALLOCATION(J)
  Let threshold value, t
  $A \leftarrow$ List of Space% of agents having Space%>80% or Space%<40%
  $\bar{A} \leftarrow$ List of Space% of agents having 40%<Space%<80%
  for $i \leftarrow 1$ **to** N **do**
    if $d_i/t \geq 0.8$ or $d_i/t \leq 0.4$ **then**
      $A_i \leftarrow \{E_i, (d_i/t) * 100\}$ appended to list
    else
      $\bar{A}_i \leftarrow \{E_i, (d_i/t) * 100\}$ appended to list
    end if
  end for
  $(A') \leftarrow ddme_server(A)$
  $(J') \leftarrow A' + \bar{A}$, Concatenating lists of ddme and non-ddme, along with new data sizes
  return J'
end procedure

FIGURE 5.2: Data Reallocation Algorithm.

Algorithm 2 Dynamic Decision Making Engine Algorithm

Input: List of Space% of agents having Space%>80% or Space%<40%, $A = \{A_1, ..., A_N\}$

Output: Updated List of optimal Space% for respective agents, $A' = \{A'_1, ..., A'_N\}$

procedure DDME_SERVER(A)
  $List_80 \leftarrow$ Sorted list of elements of A having Space%>80%, in descending order.
  $List_40 \leftarrow$ Sorted list of elements of A having Space%<40%, in increasing order.
  if length of $List_40$ > length of $List_80$ **then**
    $max_limit \leftarrow$length of $List_80$
  else
    $max_limit \leftarrow$length of $List_40$
  end if
  for $i \leftarrow 1$ **to** max_limit **do**
    $value_80 \leftarrow List_80[i] - 80$
    $value_40 \leftarrow 40 - List_40[i]$
    $List_80[i] \leftarrow List_80[i] - value_80 - value_40$
    $List_40[i] \leftarrow List_40[i] + value_40 + value_40$
  end for
  $A' \leftarrow List_80 + List_40$
  return A'
end procedure

FIGURE 5.3: Dynamic Decision Making Engine Algorithm.

Cluster 1 (15 Edge Server)			
More than 80%		Less than 40%	
Edge ID	**% Resource Utilization**	**Edge ID**	**% Resource Utilization**
Edge 10	92	Edge 1	12
Edge 2	91	Edge 6	18
Edge 7	89	Edge 9	25
Edge 5	82	Edge 4	30
Edge 3	81	Edge 8	35

FIGURE 5.4: Example of pairing edge nodes based on resource utilization.

```
DDME Agent List:
 10
[{'edge_id': 1, 'Space%': 12.0},
 {'edge_id': 2, 'Space%': 91.0},
 {'edge_id': 3, 'Space%': 81.0},
 {'edge_id': 4, 'Space%': 30.0},
 {'edge_id': 5, 'Space%': 82.0},
 {'edge_id': 6, 'Space%': 18.0},
 {'edge_id': 7, 'Space%': 89.0},
 {'edge_id': 8, 'Space%': 35.0},
 {'edge_id': 9, 'Space%': 25.0},
 {'edge_id': 10, 'Space%': 92.0}]
```

Here we can observe that the DDME Agent List consists of 10 out of 15 edge agents, which have their Utilisation percentage more than 80% (over utilised) or less than 40%(under utilised).

```
Number of edge agents with >80%:  5
[{'edge_id': 10, 'Space%': 92.0},
 {'edge_id': 2, 'Space%': 91.0},
 {'edge_id': 7, 'Space%': 89.0},
 {'edge_id': 5, 'Space%': 82.0},
 {'edge_id': 3, 'Space%': 81.0}]
```

The edge agents having more than 80% utilisation are sorted in decreasing order.

```
Number of edge agents with <40%:  5
[{'edge_id': 1, 'Space%': 12.0},
 {'edge_id': 6, 'Space%': 18.0},
 {'edge_id': 9, 'Space%': 25.0},
 {'edge_id': 4, 'Space%': 30.0},
 {'edge_id': 8, 'Space%': 35.0}]
```

The edge agents having less than 40% utilisation are sorted in increasing order.

These two lists are now used to pair the most utilised and least utilised and hence similarly for rest others.

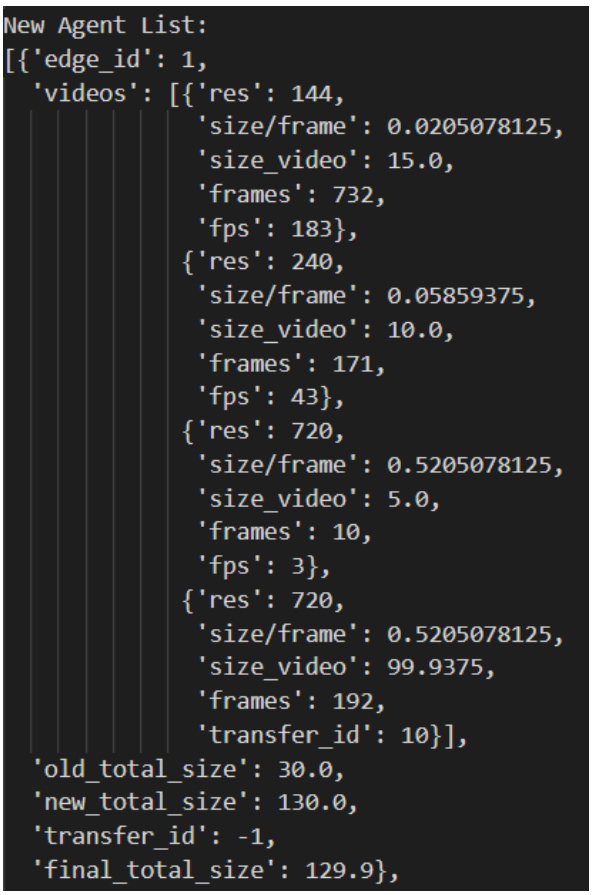
```
DDME Received List:

[{'edge_id': 10, 'Space%': 52.0, 'transfer_id': 1, 'new_total_size': 130.0},
 {'edge_id': 2, 'Space%': 58.0, 'transfer_id': 6, 'new_total_size': 145.0},
 {'edge_id': 7, 'Space%': 65.0, 'transfer_id': 9, 'new_total_size': 162.5},
 {'edge_id': 5, 'Space%': 70.0, 'transfer_id': 4, 'new_total_size': 175.0},
 {'edge_id': 3, 'Space%': 75.0, 'transfer_id': 8, 'new_total_size': 187.5},
 {'edge_id': 1, 'Space%': 52.0, 'transfer_id': -1, 'new_total_size': 130.0},
 {'edge_id': 6, 'Space%': 51.0, 'transfer_id': -1, 'new_total_size': 127.5},
 {'edge_id': 9, 'Space%': 49.0, 'transfer_id': -1, 'new_total_size': 122.5},
 {'edge_id': 4, 'Space%': 42.0, 'transfer_id': -1, 'new_total_size': 105.0},
 {'edge_id': 8, 'Space%': 41.0, 'transfer_id': -1, 'new_total_size': 102.5}]
```

After the pairing is done, data is transferred. We observe that edge agent 10 has a new utilisation of 52% which is basically lesser than the previous 92%, and the data was reallocated to edge agent 1, as clear from *transfer_id.*
transfer_id = -1 shows that no data is transferred, but instead these edge agents are receiving data.

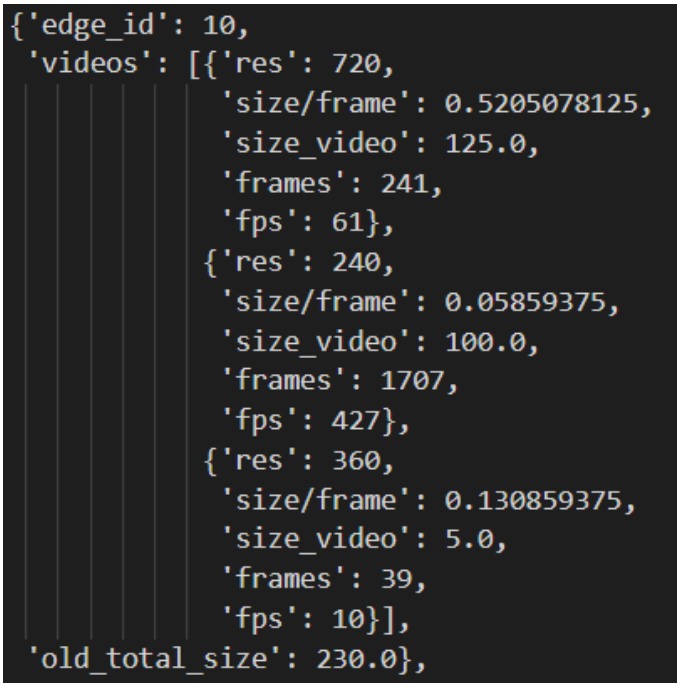
```
Old Agent List:
[{'edge_id': 1,
  'videos': [{'res': 144,
              'size/frame': 0.0205078125,
              'size_video': 15.0,
              'frames': 732,
              'fps': 183},
             {'res': 240,
              'size/frame': 0.05859375,
              'size_video': 10.0,
              'frames': 171,
              'fps': 43},
             {'res': 720,
              'size/frame': 0.5205078125,
              'size_video': 5.0,
              'frames': 10,
              'fps': 3}],
  'old_total_size': 30.0},
```

So after a single iteration of reallocation, we can see that the edge agent 1, which had just 3 video sources under it, occupying total 30MB of video data, now has:

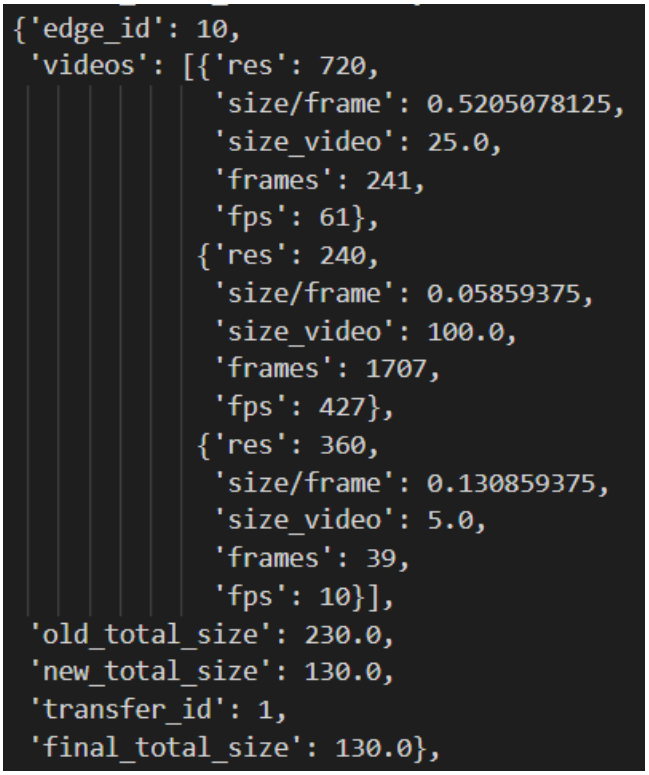


The DDME allocated videos from edge agent 10 to edge agent 1 (which can be verified by the *transfer_id*=10 parameter in the 4th video), hence a new video is being added to edge agent 1, increasing its total size to 130MB, from initial 30MB.

Edge agent 10 before reallocation:

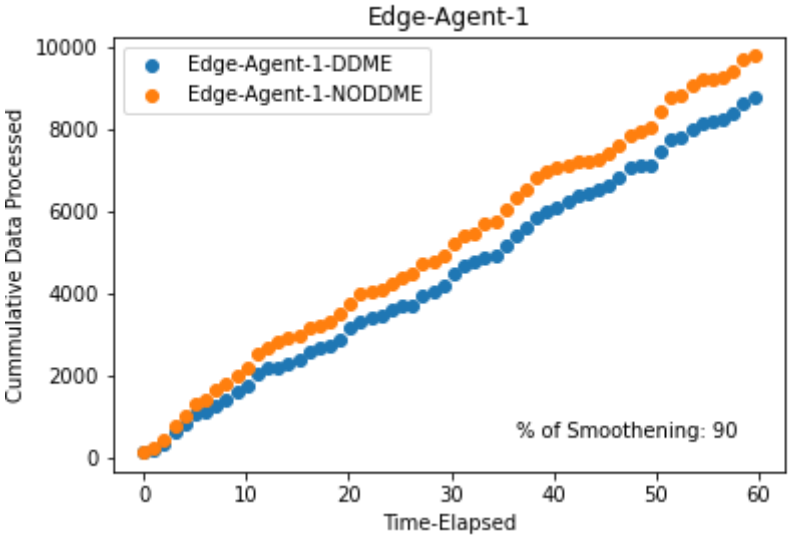


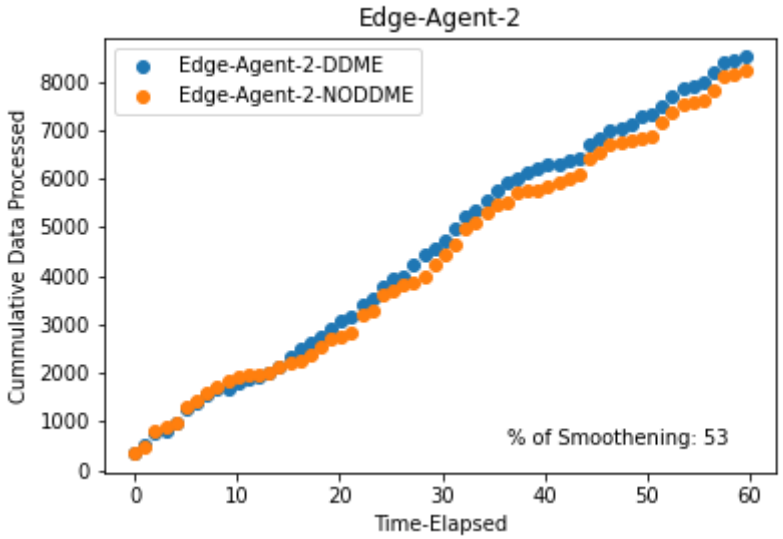
Edge agent 10 after reallocation: (transfer of 100MB to edge agent 1 is observed)

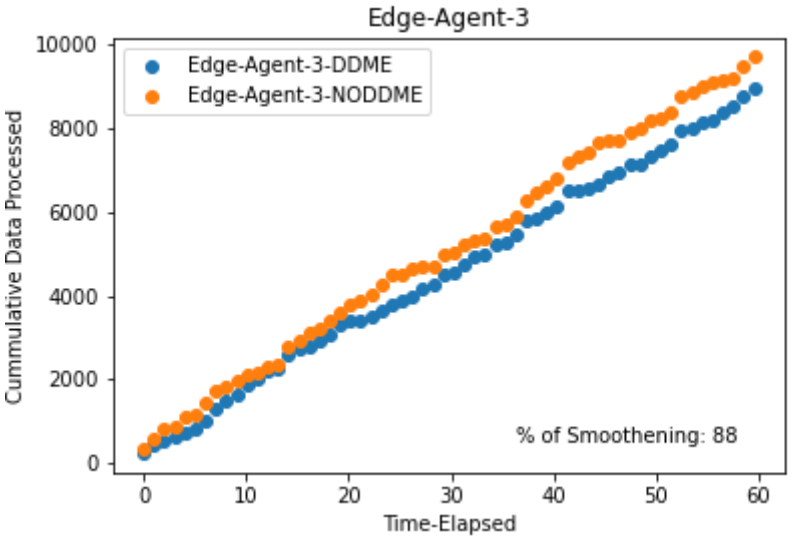


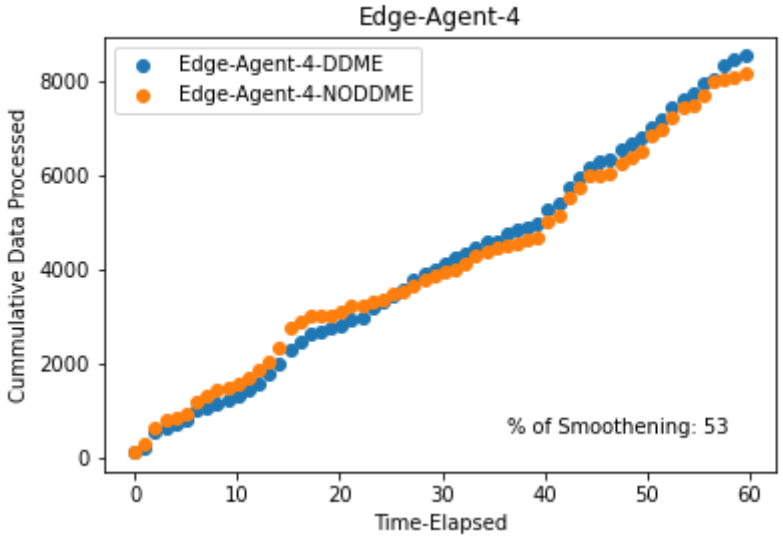
5.3 Evaluations

To get a better understanding of the above algorithm, cumulative graphs between data processed by different edge servers and time elapsed are plotted in Fig 2. Cumulative data processed against time elapsed tells us the overall load edge server has undergone and helps to track the overall usage and resource utilization on the server. The same set of values of different system parameters is passed to the same server without the DDME algorithm and with the DDME algorithm.

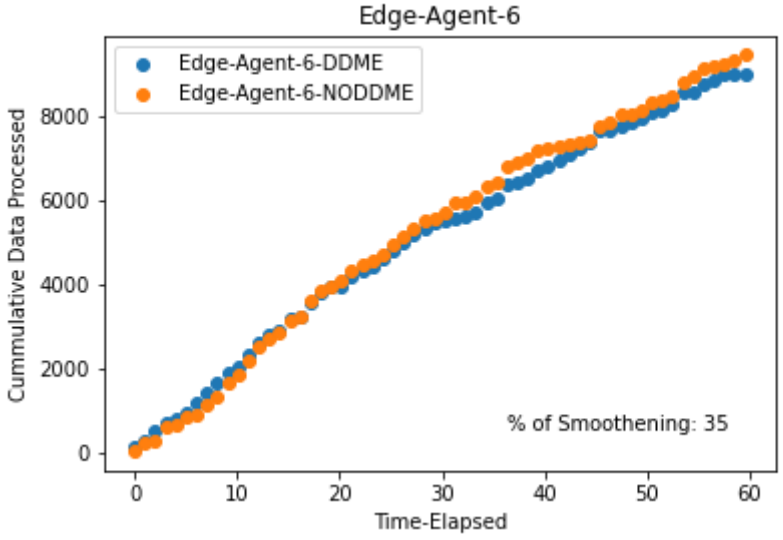

Edge-Agent-1
Edge-Agent-1-DDME
Edge-Agent-1-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 90

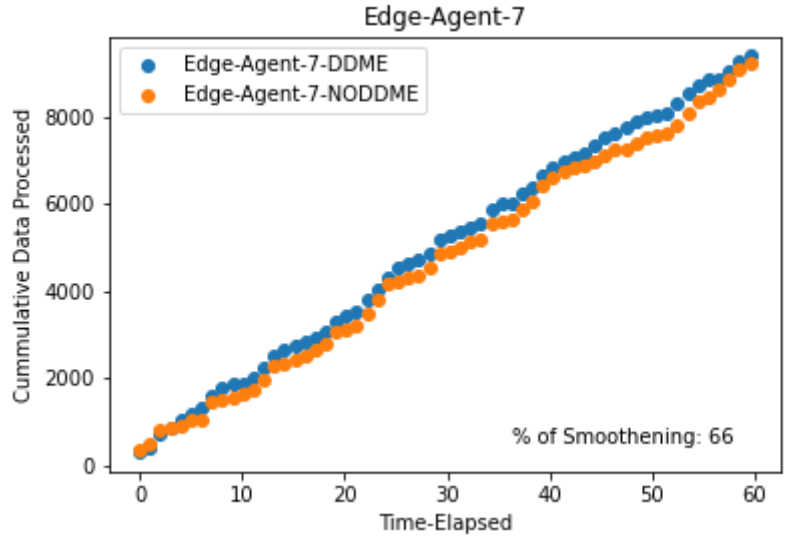

Edge-Agent-2
Edge-Agent-2-DDME
Edge-Agent-2-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 53

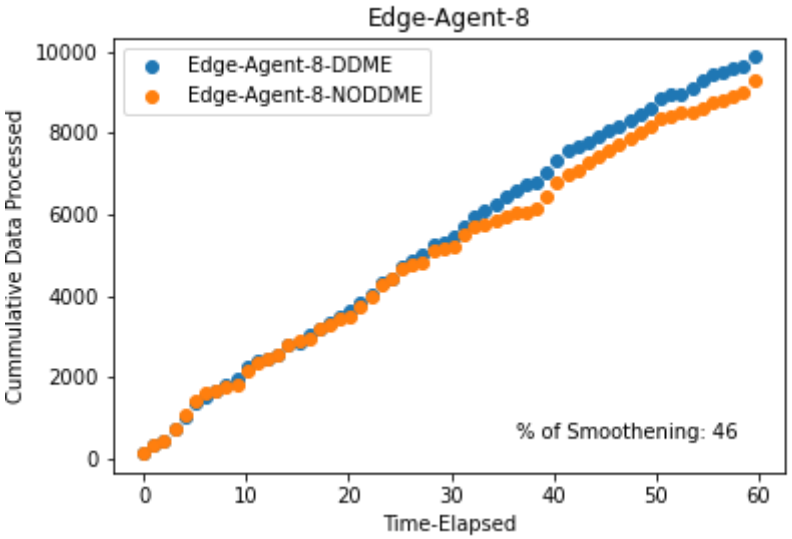

Edge-Agent-3
Edge-Agent-3-DDME
Edge-Agent-3-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 88

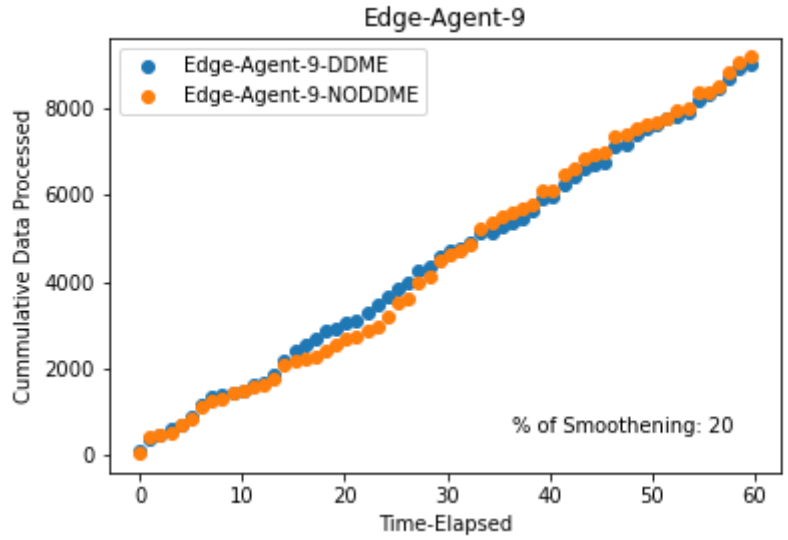

Edge-Agent-4
Edge-Agent-4-DDME
Edge-Agent-4-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 53
0
2000
4000
6000
8000
0
10
20
30
40
50
60

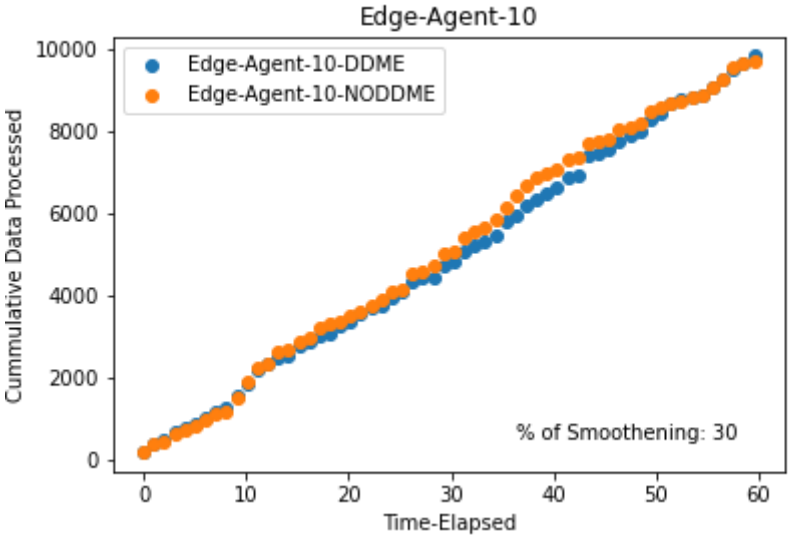

Edge-Agent-5
Edge-Agent-5-DDME
Edge-Agent-5-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 1
0
2000
4000
6000
8000
0
10
20
30
40
50
60

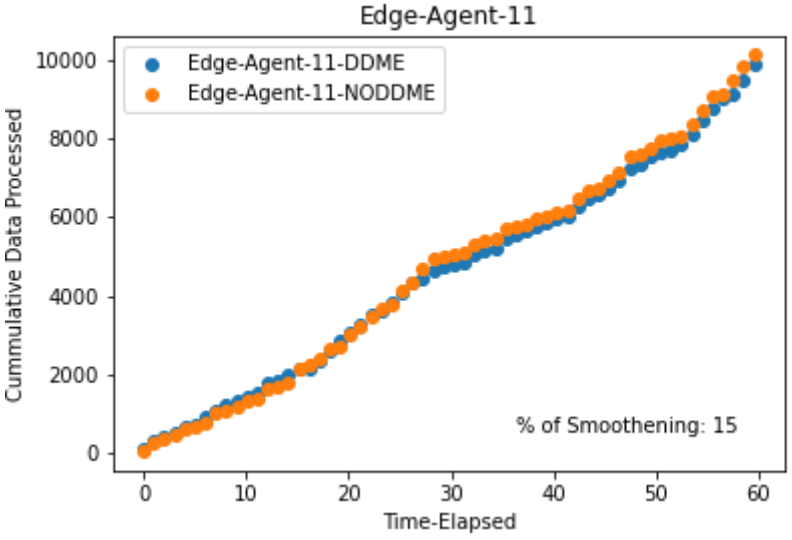

Edge-Agent-6
Edge-Agent-6-DDME
Edge-Agent-6-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 35
0
2000
4000
6000
8000
0
10
20
30
40
50
60

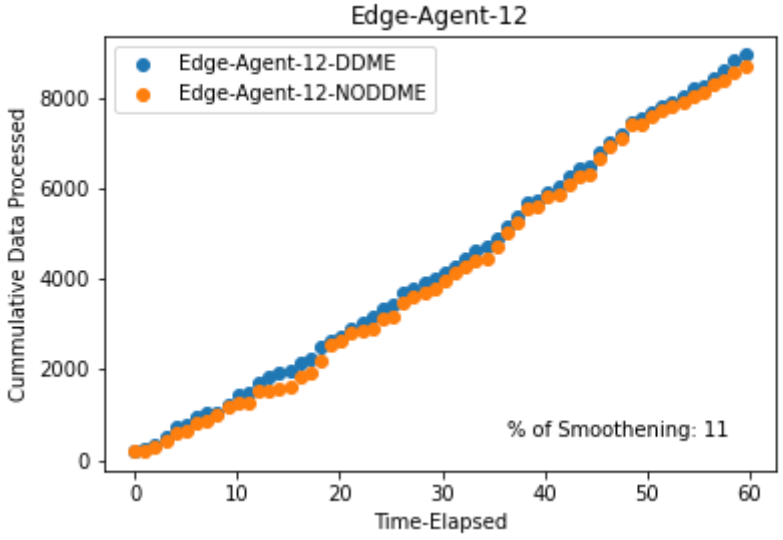

Edge-Agent-7
Edge-Agent-7-DDME
Edge-Agent-7-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 66


Edge-Agent-8
Edge-Agent-8-DDME
Edge-Agent-8-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 46


Edge-Agent-9
Edge-Agent-9-DDME
Edge-Agent-9-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 20


Edge-Agent-10
Edge-Agent-10-DDME
Edge-Agent-10-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 30
0
2000
4000
6000
8000
10000
0
10
20
30
40
50
60


Edge-Agent-11
Edge-Agent-11-DDME
Edge-Agent-11-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 15
0
2000
4000
6000
8000
10000
0
10
20
30
40
50
60


Edge-Agent-12
Edge-Agent-12-DDME
Edge-Agent-12-NODDME
Cummulative Data Processed
Time-Elapsed
% of Smoothening: 11
0
2000
4000
6000
8000
0
10
20
30
40
50
60

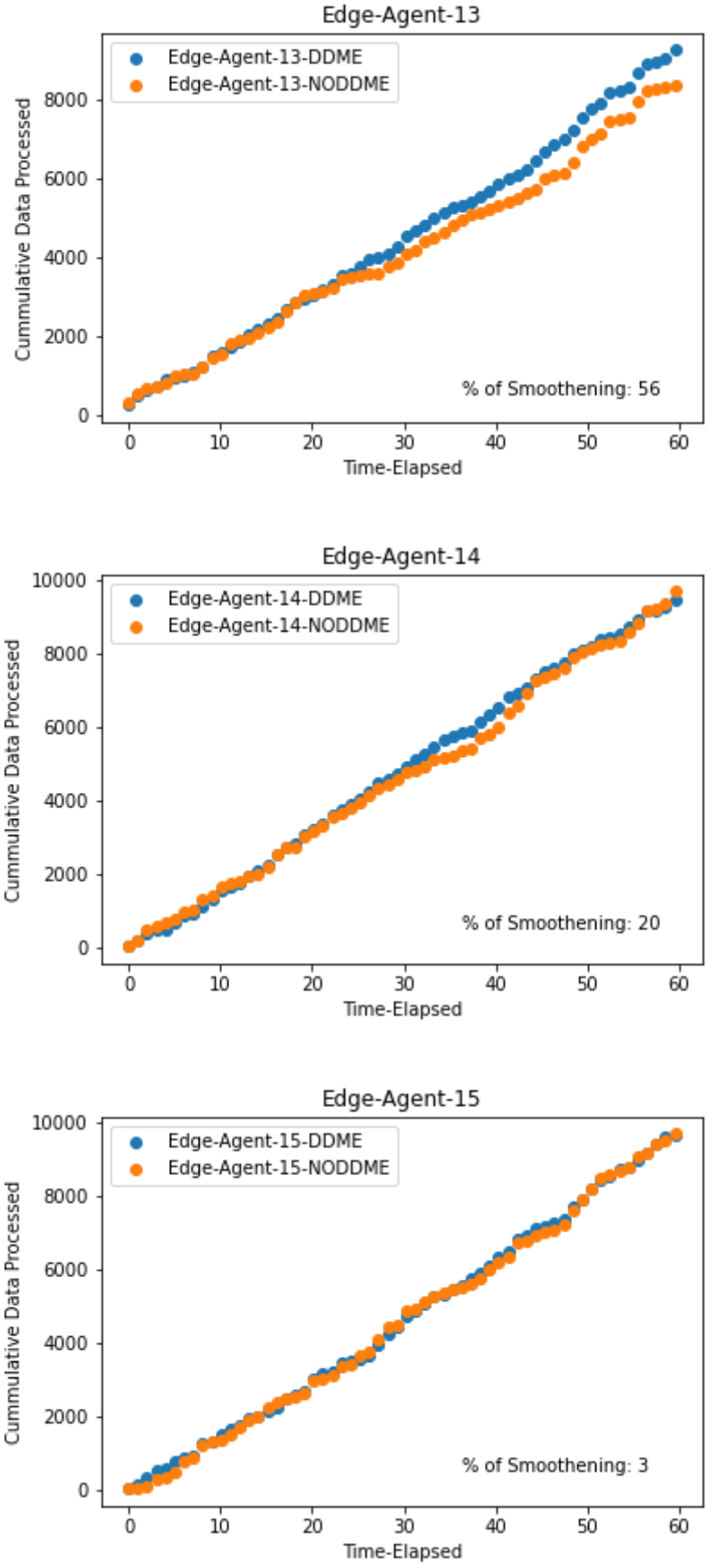


FIGURE 5.5: Optimization of edge nodes with DDME.

We can observe that for 50 run cycles of the framework, the graphs are plotted for all 15 edge agents. % of smoothening refers to how much deviation is observed in the NO-DDME plot compared to the DDME plot. For points of NO-DDME lower than the DDME, it gives us information implying the edge agent under consideration is underutilized if DDME is not used, and the free space available is wasted, whereas it could have benefitted another edge server that was overloaded, and that edge server could have sent frames to the former one, hence reducing the load on the latter one. As seen in the figure, DDME balances the load on different edge servers which are present in the system while without DDME, servers experience high load at times and there are times when they are underutilized. Sanghong Ahn et al. devised a cost-aware resource management method for video analytics applications at the edge server, which collaborates with the cloud server and includes job load balancing and task offload scheduling [8]. Their approach uses a single edge server handling multiple jobs which have subdivisions. We compared our results with their approach and found their emphasis on service latency, which is a major parameter in their decision switching between edge-cloud, as not playing a significant factor. We evaluated our proposed algorithms based on a Monte Carlo simulation with 1 million random cases and obtained an average service latency of 404ms, and Fig 4. shows the comparisons between our algorithm performance and the algorithm proposed by [8].

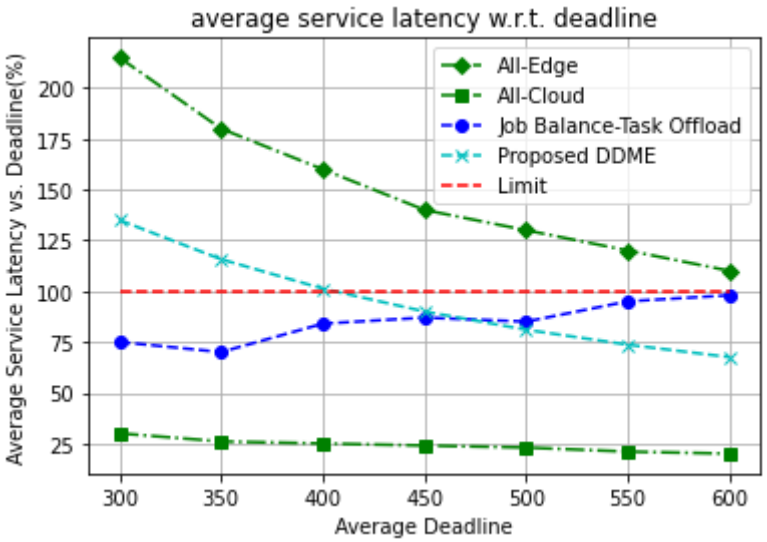
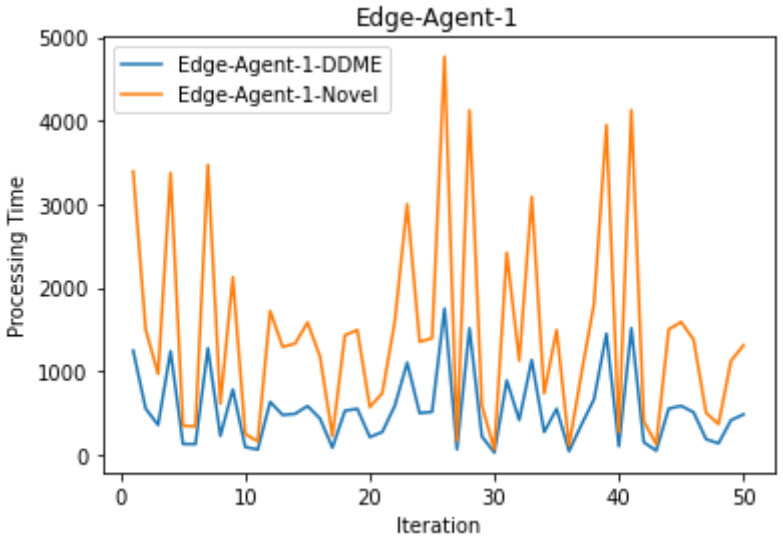


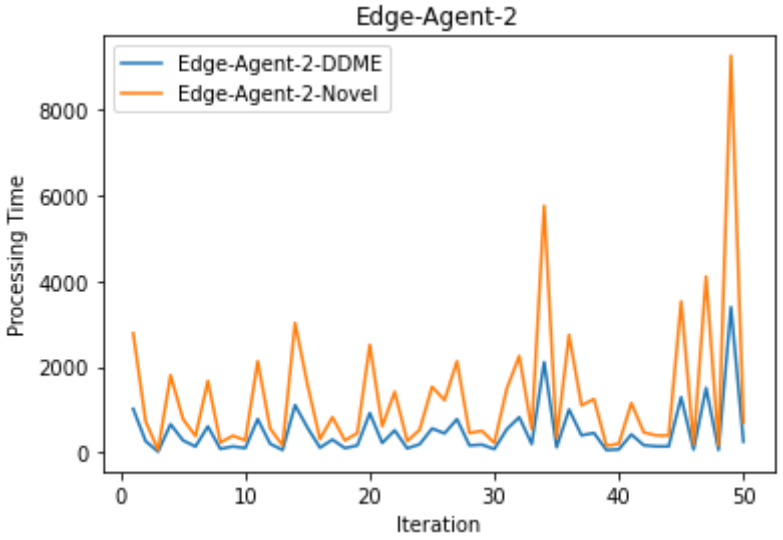
FIGURE 5.6: Performance evaluation for average service latency w.r.t. deadline.

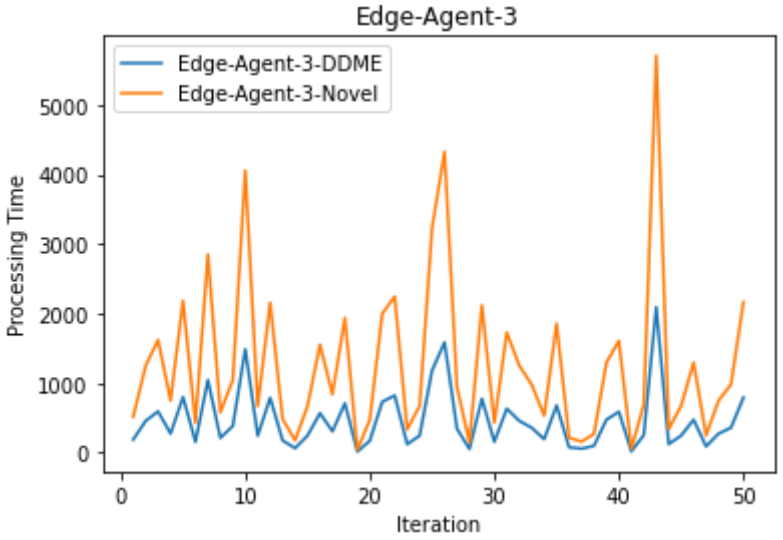
Fig. 4 shows the average service latency with varying average deadlines (300ms to 600 ms for a job). It should be noted that the average service latency on the y axis

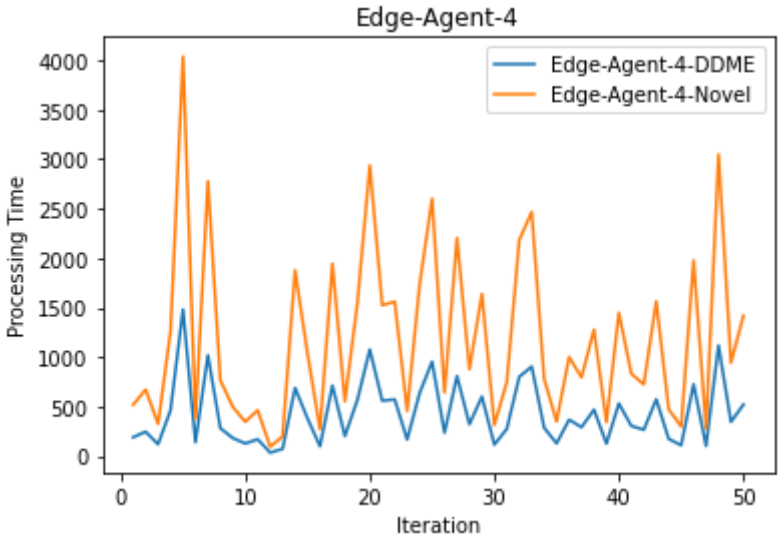
represents the ratio between the achieved service latency and the deadline requirement. Jobs with a deadline of fewer than 400 milliseconds are not met by All-Edge. This is due to the fact that the computational resources on the edge server are insufficient to process several jobs within such a short timeframe. However, while All-Cloud outperforms the other schemes in terms of average service latency, it comes at the most expensive price. Our algorithm with an average service latency of 404ms out of 1 million cases, works well within the latency deadline ranges and achieves good performance without violating deadline constraints as well as manages to do so without employing cloud services which implies no monetary costs.

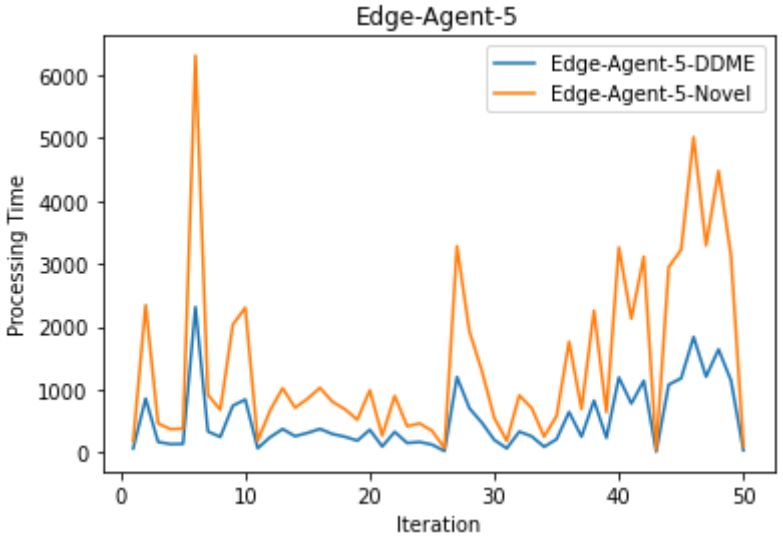
We also measured the processing time required by the novel approach algorithm and our DDME approach. For this, we considered each video frame as a ”job” and Object Detection application as the “task”. Processing time at each agent was calculated, and we found a novel approach to have completed object detection processing of all the video frames at an edge agent with 0.6 seconds per frame, which is roughly close to its upper bound deadline. However, video frames at edge agent after DDME reallocation accomplished the same feat at an astounding 0.22 seconds per frame using OpenCV i.e. 3x faster. Fig 3 compares the application of both algorithms on every edge agent for 50 iterations of runtime. The vertical gap displays the average processing time difference in each iteration run of the algorithms on an edge agent. As the DDME continuously receives video frames and reallocates them, the number of frames processed during each iteration is dynamic, hence the average processing time value is different, and this has been calculated and plotted for each iteration.

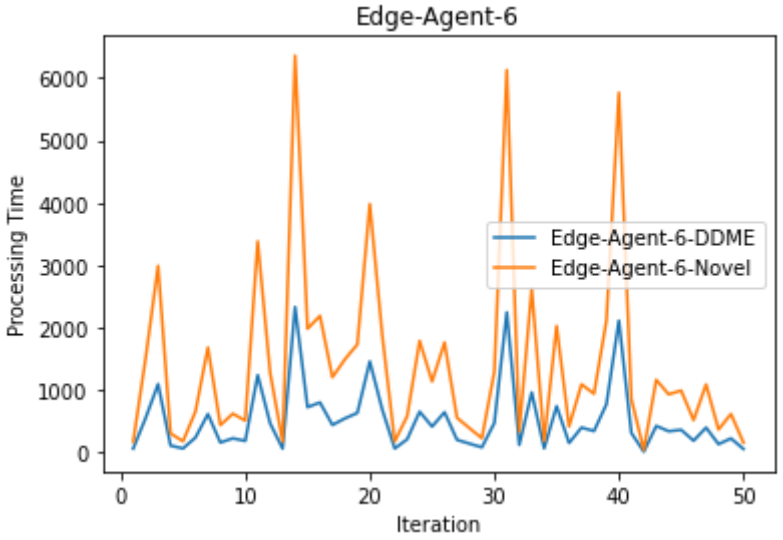

Edge-Agent-1
Edge-Agent-1-DDME
Edge-Agent-1-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
0
10
20
30
40
50

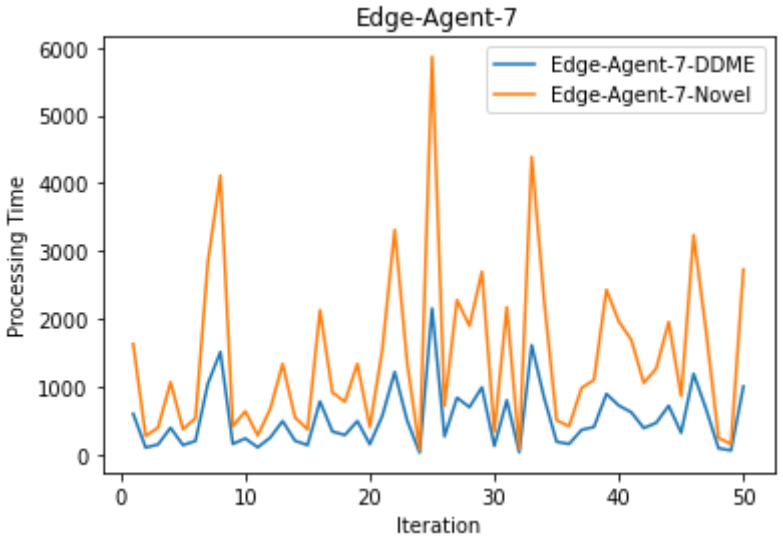

Edge-Agent-2
Edge-Agent-2-DDME
Edge-Agent-2-Novel
Processing Time
Iteration
0
2000
4000
6000
8000
0
10
20
30
40
50

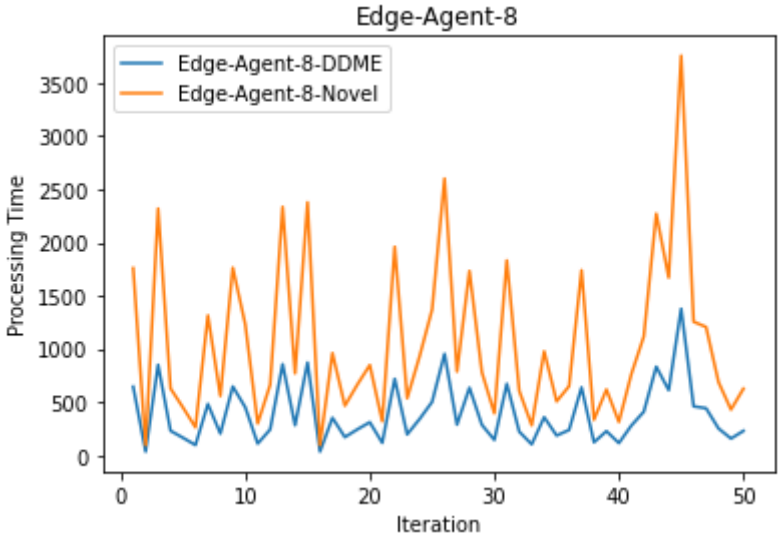

Edge-Agent-3
Edge-Agent-3-DDME
Edge-Agent-3-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
0
10
20
30
40
50

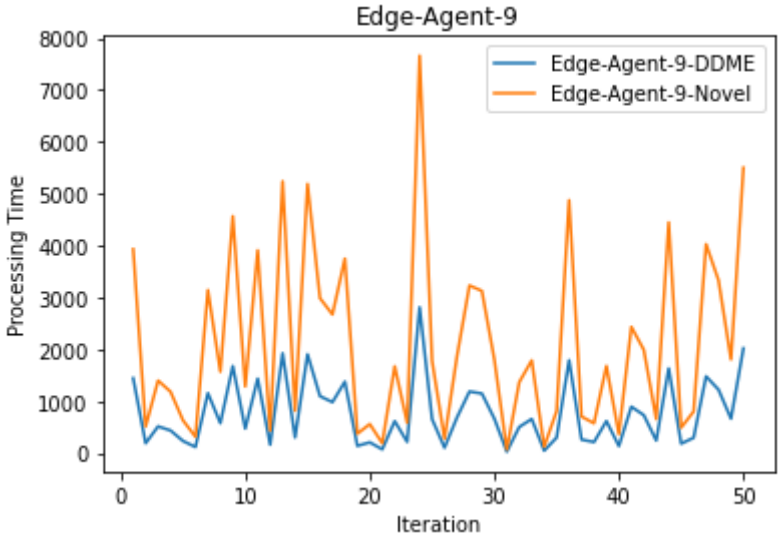

Edge-Agent-4
Edge-Agent-4-DDME
Edge-Agent-4-Novel
Processing Time
Iteration
0
500
1000
1500
2000
2500
3000
3500
4000
0
10
20
30
40
50

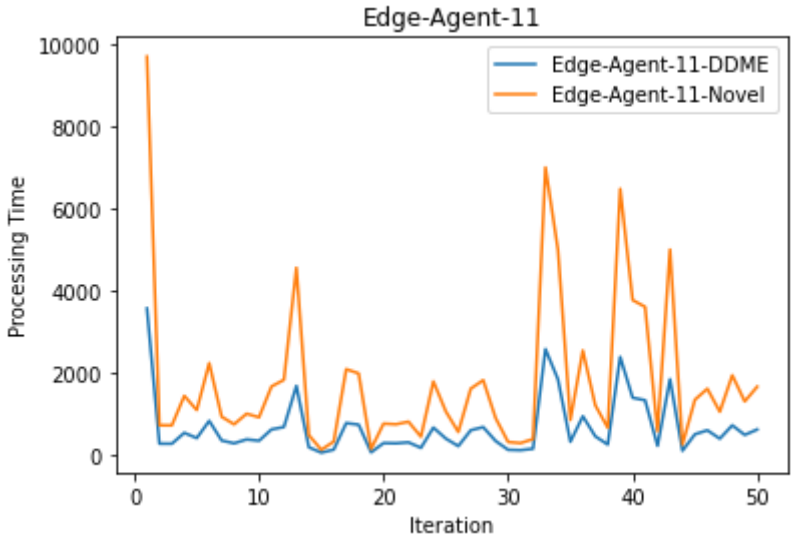

Edge-Agent-5
Edge-Agent-5-DDME
Edge-Agent-5-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40
50

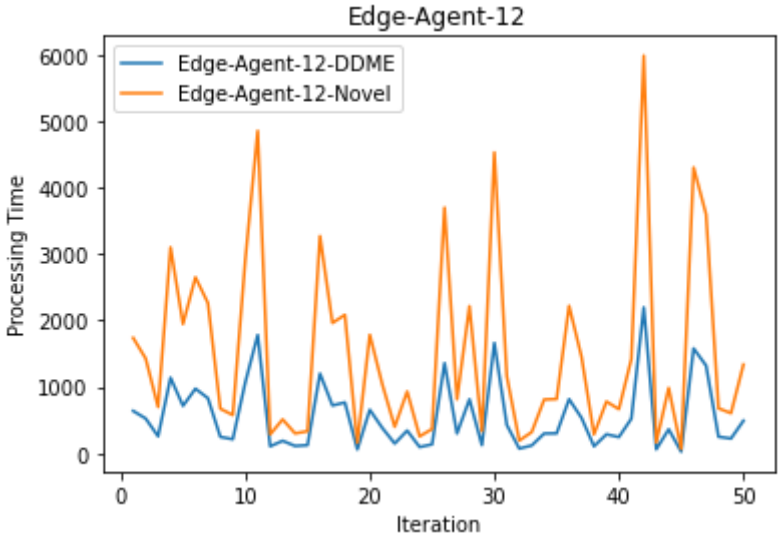

Edge-Agent-6
Edge-Agent-6-DDME
Edge-Agent-6-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40
50

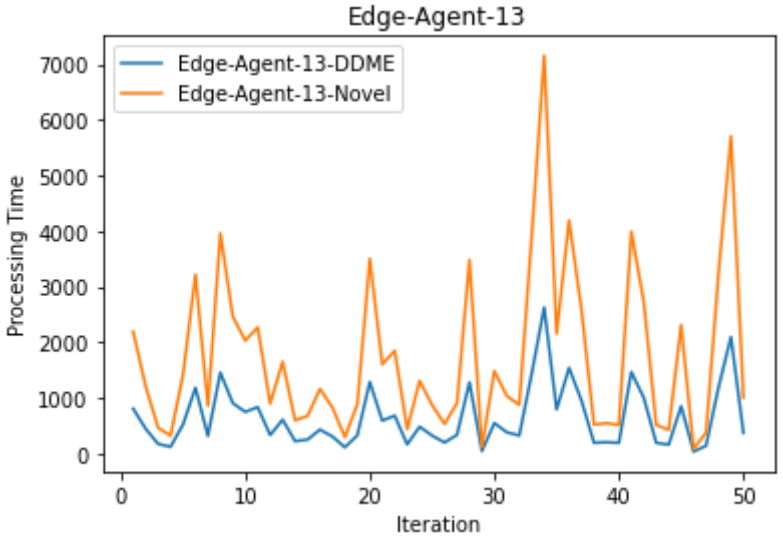

Edge-Agent-7
Edge-Agent-7-DDME
Edge-Agent-7-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40
50


Edge-Agent-8
Edge-Agent-8-DDME
Edge-Agent-8-Novel
Processing Time
Iteration
0
500
1000
1500
2000
2500
3000
3500
0
10
20
30
40
50


Edge-Agent-9
Edge-Agent-9-DDME
Edge-Agent-9-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
7000
8000
0
10
20
30
40
50


Edge-Agent-11
Edge-Agent-11-DDME
Edge-Agent-11-Novel
Processing Time
Iteration
0
2000
4000
6000
8000
10000
0
10
20
30
40
50


Edge-Agent-12
Edge-Agent-12-DDME
Edge-Agent-12-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
0
10
20
30
40
50


Edge-Agent-13
Edge-Agent-13-DDME
Edge-Agent-13-Novel
Processing Time
Iteration
0
1000
2000
3000
4000
5000
6000
7000
0
10
20
30
40
50

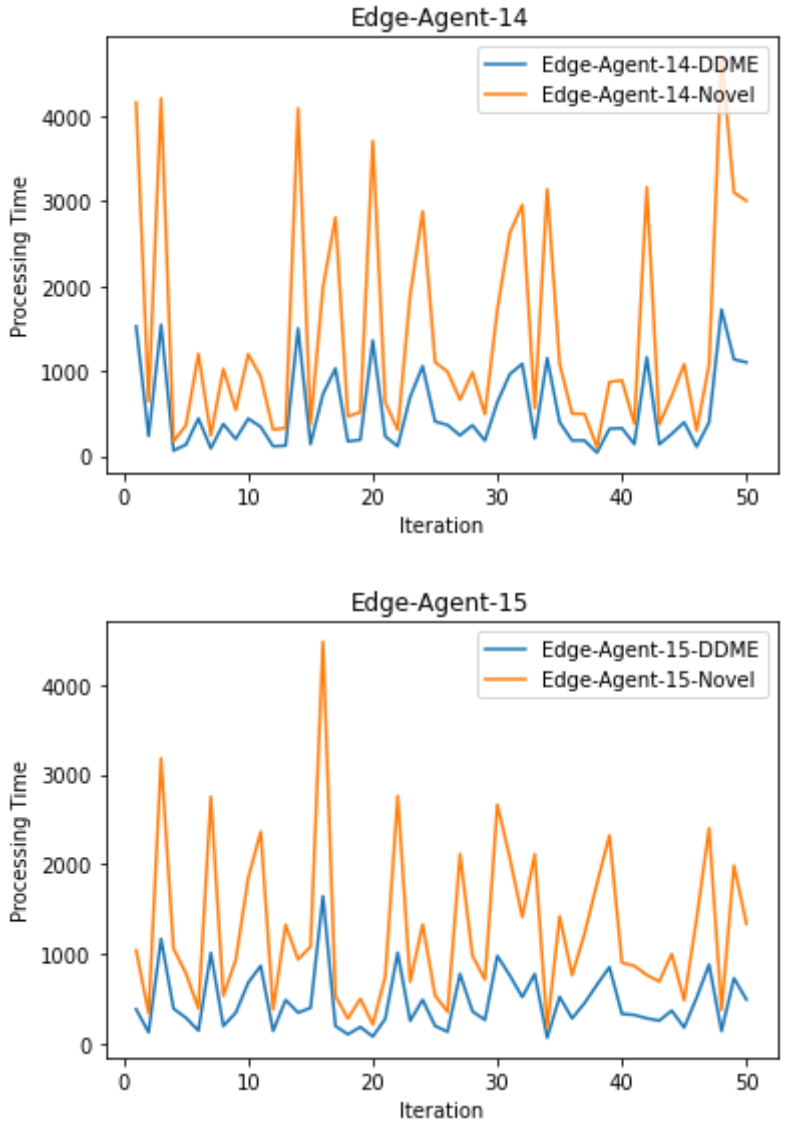


FIGURE 5.7: Comparison of DDME with the framework proposed by Ahn et al.

5.4 Summary

In this chapter, we have discussed our proposed DDME Algorithm and this architecture, and how it performs compared with the Task Offloading-Job Balancing algorithm proposed by Sanghong Ahn et al.

The algorithm was successfully reallocating the load on the edge servers dynamically and the image processing was performed faster in DDME architecture compared to Task Offloading-Job Balancing algorithm.

We also discussed the evaluations of the algorithms which were based on a Monte Carlo simulation and compared the situations where a DDME architecture was used, and another where the architecture was not employed, and their performances were plotted.

CHAPTER 6:
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

With the above framework, it is observed that the DDME algorithm helps in a smoother video processing which has two-fold benefits i.e. effective utilization of all resources in the cluster and cost saving. As DDME helps in shifting the load to a different edge server as per availability, there is no requirement for augmentation of extra memory and CPU in the cloud infrastructure which normally attracts higher costs from the cloud service provider. Also as DDME provides freedom to choose different threshold values for different edge servers as per their computational power, users can use the existing infrastructure and there is no need to invest in newer infrastructure at once. This will be a huge advantage for remote areas in third world countries which have some low computational power infrastructure.

Before we started analysis of the various algorithms, we intended to achieve certain objectives through the DDME algorithm, and we have been able to accomplish the following objectives:

1. Network topology and pipeline of video analytics applications simplified: The DDME framework adds upon the existing layout to balance the load on resources of the edge server to an optimum capacity.
2. Caching, Computing and Communication challenges, the Edge C-3 balanced: Dynamic reallocation every 5 seconds assured the caching problem was solved, the DDME logic of utilisation simplified computations and lastly, lack of cloud servers assured there was no extra latency factor responsible in slowing down real time applications.
3. Cooperative processing on resource-abundant edge devices: Effective data transfer between edge servers reduced processing times drastically, hence computational resources were saved, which can be used for other heavy processing tasks.
4. Real time processing and efficient resource management: Avoiding cloud servers killed two birds with one stone: it helped in uninterrupted

real time runtime as transmission times were not under consideration, also cloud services are expensive from current cloud providers.

5. Deadline-driven scientific workflow: DDME is able to perform better than Task Offload-Load Balancing algorithm, for same videos and object detection under the deadline constraints of [300, 600]ms.
6. Reducing cost of cloud resources: Expense on cloud has been removed. Cost saved here can be used for improving edge server capacity.

6.2 Future Scope

The same algorithm can be used as one of the filters in existing neural networks to improve the overall video processing. Similarly, DDME can take up tracking and monitoring activities also along with load balancing and based on the situation can do pattern matching under normal conditions and help in anomaly detection.

Future work on DDME should try to achieve the following objectives

1. Faster AI employment in edge computing networks.
2. Improving Quality of Experience (QoE).

Currently YOLO has been used for object detection, but there is scope for lesser computational heavy machine learning algorithms, which also improve decision making of the algorithm. Certain simple OpenCV algorithms are lite and lesser burden on the architecture, although compromise on accuracy can be observed. A tradeoff between these two must be found, and the statistical calculations used currently can be replaced by efficient machine learning algorithms, which will improve the dynamic nature of the reallocation.

REFERENCES

[1] Qingyang Zhang, Hui Sun, Xiaopei Wu, and Hong Zhong. Edge video analytics for public safety: A review. Proceedings of the IEEE, 107(8):1675–1696, 2019. doi: 10.1109/JPROC. 2019.2925910.

[2] Behrouz Jedari, Gopika Premsankar, Gazi Illahi, Mario Di Francesco, Abbas Mehrabi, and Antti Yl¨a-J¨a¨aski. Video caching, analytics, and delivery at the wireless edge: A survey and future directions. IEEE Communications Surveys Tutorials, 23(1):431–471, 2021. doi: 10.1109/COMST.2020.3035427.

[3] Zhi Zhou, Xu Chen, En Li, Liekang Zeng, Ke Luo, and Junshan Zhang. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proceedings of the IEEE, 107(8):1738–1762, 2019. doi: 10.1109/ JPROC.2019.2918951.

[4] Changchun Long, Yang Cao, Tao Jiang, and Qian Zhang. Edge computing framework for cooperative video processing in multimedia iot systems. IEEE Transactions on Multimedia, 20(5):1126–1139, 2018. doi: 10.1109/TMM. 2017.2764330.

[5] Li Lin, Xiaofei Liao, Hai Jin, and Peng Li. Computation offloading toward edge computing. Proceedings of the IEEE, 107(8): 1584–1607, 2019. doi: 10.1109/JPROC.2019. 2922285.

[6] Xukan Ran, Haolianz Chen, Xiaodan Zhu, Zhenming Liu, and Jiasi Chen. Deep Decision: A mobile deep learning framework for edge video analytics. pages 1421–1429, 2018. doi: 10.1109/INFOCOM.2018.8485905.

[7] Jyoti Sahni and Deo Prakash Vidyarthi. A cost-effective deadline-constrained dynamic scheduling algorithm for scientific workflows in a cloud environment. IEEE Transactions on Cloud Computing, 6(1):2–18, 2018. doi: 10.1109/TCC.2015.2451649.

[8] Sanghong Ahn, Joohyung Lee, Tae Yeon Kim, and Jun Kyun Choi. A novel edge-cloud interworking framework in the video analytics of the internet of things. IEEE Communications Letters, 24(1):178–182, 2020. doi: 10.1109/ LCOMM.2019.2943857.

[9] Ala Al-Fuqaha, Mohsen Guizani, Mehdi Mohammadi, Mohammed Aledhari, and Moussa Ayyash. Internet of things: A survey on enabling technologies, protocols, and applications. IEEE Communications Surveys Tutorials, 17(4):2347–2376, 2015. doi: 10.1109/COMST.2015.2444095.

[10] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, "Edge Computing: Vision and

Challenges," in IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[11] W. Shi and S. Dustdar, "The Promise of Edge Computing," in Computer, vol. 49, no. 5, pp. 78-81, May 2016, doi: 10.1109/MC.2016.145.

[12] G. Ananthanarayanan et al., "Real-Time Video Analytics: The Killer App for Edge Computing," in Computer, vol. 50, no. 10, pp. 58-67, 2017, doi: 10.1109/MC.2017.3641638.

[13] A. Amjad, F. Rabby, S. Sadia, M. Patwary and E. Benkhelifa, "Cognitive Edge Computing based resource allocation framework for Internet of Things," 2017 Second International Conference on Fog and Mobile Edge Computing (FMEC), 2017, pp. 194-200, doi: 10.1109/FMEC.2017.7946430

[14] Xukan Ran, Haoliang Chen, Zhenming Liu, and Jiasi Chen. 2017. Delivering Deep Learning to Mobile Devices via Offloading. In Proceedings of the Workshop on Virtual Reality and Augmented Reality Network(VR/AR Network '17). Association for Computing Machinery, New York, NY, USA, 42–47. DOI:https://doi.org/10.1145/3097895.3097903

[15] Tiffany Yu-Han Chen, Hari Balakrishnan, Lenin Ravindranath, and Paramvir Bahl. 2016. GLIMPSE: Continuous, Real-Time Object Recognition on Mobile Devices. GetMobile: Mobile Comp. and Comm. Volume 20, Issue 1 (January 2016), 26–29. DOI:https://doi.org/10.1145/2972413.2972423

[16] Mobile-Edge Computing – Introductory Technical White Paper. Authored by the founders of the Mobile-edge Computing (MEC) industry initiative:
Huawei: Milan Patel, Yunchao Hu, Patrice Hédé, IBM: Jerome Joubert, Chris Thornton, Brian Naughton, Intel: Julian Roldan Ramos, Caroline Chan, Valerie Young, Soo Jin Tan, Daniel Lynch, Nokia Networks: Nurit Sprecher, Torsten Musiol, Carlos Manzanares, Uwe Rauschenbach, NTT DOCOMO: Sadayuki Abeta, Lan Chen, Kenji Shimizu , Vodafone: Adrian Neal, Peter Cosimini, Adam Pollard, Guenter Klas. Published in September 2014.

[17] Google. (n.d.). *Recommended upload encoding settings - YouTube Help*. Google.

[18] Omni Calculator - *Streaming Bitrate Calculator*
https://www.omnicalculator.com/other/streaming-bitrate

[19] Bitrate vs. Resolution for Professional Video Broadcasting | Dacast, 2022
https://www.dacast.com/blog/bitrate-vs-resolution/

[20] What Is a Good Upload Speed for Live Streaming, 2022
https://restream.io/blog/what-is-a-good-upload-speed-for-streaming/

Cost effective edge computing

ORIGINALITY REPORT

23%	16%	19%	11%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Sanghong Ahn, Joohyung Lee, Tae Yeon Kim, Jun Kyun Choi. "A Novel Edge-Cloud Interworking Framework in the Video Analytics of the Internet of Things", IEEE Communications Letters, 2020 Publication	5%
2	www.cs.ucr.edu Internet Source	3%
3	Submitted to Visvesvaraya National Institute of Technology Student Paper	2%
4	mafiadoc.com Internet Source	1%
5	csdl2.computer.org Internet Source	1%
6	dblp.dagstuhl.de Internet Source	1%
7	xml.jips-k.org Internet Source	1%